

Fast and Expressive Multi-Byte Prediction with Probabilistic Circuits

Andreas Grivas¹ Lorenzo Loconte¹ Emile van Krieken² Piotr Nawrot¹ Yu Zhao¹ Euan Wielewski³
Pasquale Minervini^{1,4} Edoardo Ponti^{1*} Antonio Vergari^{1*}

Abstract

Multi-token prediction (MTP) is a prominent strategy to significantly speed up generation in large language models (LLMs), especially in byte-level LLMs, which are tokeniser-free but prohibitively slow. However, many existing MTP methods either assume independence between future tokens, sacrificing expressiveness, or generate tokens one at a time within the window, increasing latency. In this work, we investigate the trade-off between expressiveness and latency in MTP within the framework of probabilistic circuits (PCs). Our framework, MTPC, allows one to explore different ways to encode the *joint* distributions over future tokens by selecting circuit architectures, generalising classical models such as (hierarchical) mixture models, hidden Markov models, and tensor networks. We show the efficacy of MTPC by retrofitting existing byte-level LLMs, such as EvaByte, and byte-fied subword models, such as Llama3.2 3B. Our experiments show that, when combined with speculative decoding, MTPC substantially speeds up generation compared to MTP with independence assumptions, while guaranteeing to retain the performance of the original verifier LLM. We also rigorously study the optimal trade-off between expressiveness and latency when exploring the possible parameterisations of MTPC, such as PC architectures and partial layer sharing between the verifier and draft LLMs.¹

1. Introduction

Autoregressive (AR) large language models (LLMs) can only perform single-token prediction (STP) as they generate one token at a time, incurring significantly high latency, en-

^{*}Joint last authors. ¹School of Informatics, University of Edinburgh, UK ²Vrije Universiteit Amsterdam, NL ³NatWest Group ⁴Miniml.AI. Correspondence to: Andreas Grivas <agri-vas@ed.ac.uk>.

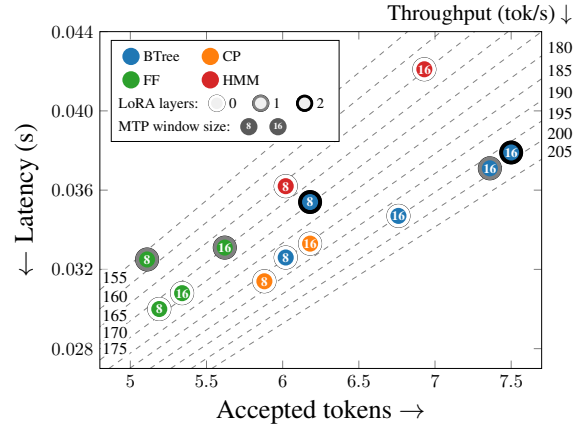


Figure 1. MTPC allows us to trade-off efficiency (latency) and expressiveness (token acceptance) with different MTP designs by choosing 1) the probabilistic circuit (PC) architecture (FF, CP, HMM, BTree); 2) the number of layers shared between draft and verifier models in self-speculative sampling. Dotted lines indicate iso-throughput (bytes/sec) regions, highlighting designs such as BTree for 16 bytes and 1 LoRA layer that achieve the best throughput for our retrofitted EvaByte-MTP model (Sec. 4).

ergy demand, and deployment costs. This problem affects subword models, but is dramatically exacerbated in byte-level ones (Minixhofer et al., 2025; Wang et al., 2024; Yu et al., 2023, *inter alia*) as many more decoding steps are required to generate a text of the same length. Among possible alternatives to speed up generation (Ankner et al., 2024; DeepSeek-AI et al., 2024; Nawrot et al., 2023; Pagnoni et al., 2025; Łańcucki et al., 2026), multi-token prediction (MTP) stands out as it promises to predict a window of multiple tokens *all at once*, may they be subwords (Gloeckle et al., 2024; Cai et al., 2024) or bytes (Gloeckle et al., 2024; Zheng et al., 2025). As such, MTP LLMs can achieve a significantly higher throughput than STP ones, as they decrease the number of passes through the LLM to generate the same text. While recent works on MTP focuses on subword models (Cai et al., 2024; Li et al., 2024), MTP can improve throughput the most in byte-level models (Pagnoni et al., 2025; Wang et al., 2024) due to their increased computational overhead. If one could model large sequences

¹Code and models available at github.com/april-tools/mtpc.

of future bytes, byte-level LLMs could become more mainstream as they already obviate many limitations of subword tokenizers, such as uneven efficiency (Ahia et al., 2023; Dagan et al., 2024), lack of interoperability (Minixhofer et al., 2026), and vulnerabilities (Rumbelow & Watkins, 2023; Land & Bartolo, 2024; Geiping et al., 2024).

However, modelling the joint distribution over many future bytes as tokens in a window is challenging, as it requires balancing *expressiveness*, i.e., representing all the dependencies between bytes, and *efficiency*, i.e., minimising latency. Many existing MTP approaches favour the latter by making an unrealistic assumption: they consider all future tokens to be independent (Zheng et al., 2025; Cai et al., 2024; Gloeckle et al., 2024). This clearly comes at the expense of expressiveness (Ankner et al., 2024; Wertheimer et al., 2024), as the choice of a byte for a position within the window cannot influence the probability of the others.

For example, consider the prompt: “Name a capital of South Africa”, where *Cape Town* and *Pretoria* are equally likely completions. A byte-level MTP model with independence assumptions over an 8-token window could return *Cretria* as an argmax, because replacing *P* with *C* cannot change the probability of other tokens. More concerningly, a number of “byte-salad” continuations, such as *Crptoria*, *Crpt ria* and *Crpt roa*, can also have high probability, despite having almost zero probability under the STP model. Most importantly, the number of these erroneous continuations grows exponentially with respect to the MTP window size, making the independence assumption problematic.

Perhaps the dominant paradigm for relaxing independence assumptions in MTP are shallow AR heads (Ankner et al., 2024; Li et al., 2024; DeepSeek-AI et al., 2024). However, these heads retain autoregressive generation within the prediction window, which we view as closer in spirit to accelerated AR decoding than to joint MTP. Importantly for our purposes, we seek methods that satisfy the following two *desiderata*: a) they yield a joint distribution over the token window that can be sampled from in parallel, and b) they expose the dependency structure of the joint as a modelling choice. Shallow AR heads satisfy neither. Moreover, while Hydra (Ankner et al., 2024) and EAGLE (Li et al., 2024) are state-of-the-art for subword-level MTP, their reported gains do not necessarily translate to byte-level models; a direct empirical comparison is orthogonal to our contribution and faces several technical obstacles, see Appendix D.

Satisfying our desiderata, Basharin et al. (2025) introduced dependencies into MTP using a mixture over the future token probabilities, but a single mixture can only add limited expressiveness. Crucially, understanding how to increase expressiveness while optimally trading off efficiency in a systematic way is still an open question. We address this gap by proposing an MTP framework based on probabilistic

circuits (PCs; Choi et al., 2020; Vergari et al., 2021), which we name MTPC. MTPC uses PCs to parameterise the joint distribution over future tokens as tractable computational graphs that encode hierarchical mixture models. As such, MTPC offers a way to systematically navigate the spectrum of MTP architectural variants, encompassing fully factorised models (Zheng et al., 2025; Cai et al., 2024; Gloeckle et al., 2024), shallow mixtures (Basharin et al., 2025), but also more expressive parameterisations: hidden Markov models (HMMs) and binary tree factorisations (BTrees), which are novel for MTP.

Moreover, in contrast to previous work on MTP (Zheng et al., 2025; Cai et al., 2024), MTPC guarantees we match the quality of an AR LLM via speculative decoding (Leviathan et al., 2022; Chen et al., 2023; Stern et al., 2018; Xia et al., 2024). For greedy decoding, the output is guaranteed to be identical to the AR LLM; for sampling, it matches in expectation. This shows that the throughput cost of the guarantee is not as large as previously suggested. We achieve this by sharing the LLM backbone for the draft and verifier models for different numbers of layers, highlighting how this creates a *second dimension to trade-off expressiveness* (as hidden representations between draft and verifier are allowed to differ) and *latency* (as each non-shared layer requires separate forward passes). We illustrate the two trade-offs at the core of MTPC in Fig. 1.

In summary, we make the following contributions: **C1)** we introduce MTPC, a fast MTP framework based on PCs that relaxes the independence assumptions of previous work and generalises tensor decomposition methods (Basharin et al., 2025); **C2)** we rigorously identify trade-offs between the number of accepted tokens in speculative decoding and the latency of generation, based on different choices of PC architectures and partial layer sharing; **C3)** we empirically demonstrate the effectiveness of MTPC by repurposing two byte-level LLMs, EvaByte (Zheng et al., 2025) and Llama 3.2 3B Byte (Minixhofer et al., 2025), into our framework. We find that MTPC substantially increases the throughput of EvaByte by $5.15\times$ and that of Llama by $2.24\times$ with respect to AR generation and both by $1.17\times$ with respect to MTP with independence assumptions, by supporting MTP windows of up to 16 bytes.

2. Speeding up Generation with MTP and Speculative Decoding

We now introduce MTP and speculative decoding, the two main ingredients of MTPC.² The former accelerates generation from the target LLM; the latter guarantees that its generation quality is preserved.

²We adapt notation from the tensor and circuit literature (Lofonte et al., 2025a); see also Appendix A.

MTP. A classical STP LLM encodes a distribution over sequences of tokens $\{\mathbf{x}_t\}$ defined over a vocabulary $\mathcal{V}$ as $\prod_t p(x_{t+1} \mid \mathbf{x}_{\leq t})$, where $\mathbf{x}_{\leq t}$ is the context, *i.e.* the observed tokens at timestep t . MTP (Gloeckle et al., 2024) aims to extend an STP LLM that predicts a single token at a time through $p(x_{t+1} \mid \mathbf{x}_{\leq t})$, to an MTP model, q_θ , that models the *joint* probability of a window of n future tokens and generates them *simultaneously*, *i.e.* we model

$$q_\theta(x_{t+1}, x_{t+2}, \dots, x_{t+n} \mid \mathbf{x}_{\leq t}), \quad (1)$$

where θ denotes a given parameterisation for the joint.³ The first dimension to trade-off expressiveness and efficiency in MTP pertains to compactly representing q_θ . Unlike for $p(x_{t+1} \mid \mathbf{x}_{\leq t})$, we would need to store more than a vector of logits $\mathbf{a} \in \mathbb{R}^v$ of a single univariate categorical distribution for a vocabulary size $v = |\mathcal{V}|$ for every timestep t . The most expressive, but least efficient way to do so, would be to store an n -dimensional tensor $\mathcal{A} \in \mathbb{R}^{v^{(1)} \times \dots \times v^{(n)}}$ of logits having v^n entries, but this scales exponentially in n . Next, we review past attempts to avoid storing $\mathcal{A}$ explicitly.

Fully factorised. A common way to boost efficiency is to assume all n future tokens are independent (Zheng et al., 2025; Cai et al., 2024; Gloeckle et al., 2024) and factorise the distribution q_θ in Eq. (1) as

$$\prod_{i=1}^n q_\theta(x_{t+i} \mid \mathbf{x}_{\leq t}). \quad (\text{FF})$$

This comes with the benefit that one needs to store only n probability vectors, each of dimension v , to represent the joint distribution in Eq. (1). However, as already discussed in the introduction, this severely limits the model’s expressiveness (Ankner et al., 2024; Wertheimer et al., 2024).

Canonical polyadic (CP) factorisation. Dependencies between future tokens can be recovered by introducing explicit latent variables (Lee et al., 2018). To this end, Basharin et al. (2025) propose to factorise Eq. (1) via an r -rank CP decomposition. A CP decomposition introduces one discrete latent variable, z , that encodes a mixture of r fully-factorised components, rewriting Eq. (1) as

$$\sum_{z=1}^r q(z \mid \mathbf{x}_{\leq t}) \prod_{i=1}^n q_\theta(x_{t+i} \mid z, \mathbf{x}_{\leq t}), \quad (\text{CP})$$

where $q(z \mid \mathbf{x}_{\leq t})$ are the mixture coefficients.⁴ Before showing how we can generalize both FF and CP MTP with probabilistic circuits, we review how to ensure MTP models match the quality of a given STP model.

³We drop the index t from θ_t for readability when not needed.

⁴Basharin et al. (2025) calls CP a mixture of experts (MoE), but we note this is incorrect as the weights ω_j do not depend on future tokens, but only on past ones. Moreover, while they argue that training CP is challenging and requires insights from the MoE literature, we can train them as well as deeper mixture variants easily without MoE-tailored losses (see Sec. 4).

Speculative decoding (Stern et al., 2018; Leviathan et al., 2022; Chen et al., 2023; Xia et al., 2024) can be combined with MTP to speed up generation while guaranteeing no loss in generation quality. Given a target STP LLM that we wish to accelerate, speculative decoding consists of a cycle of two steps: 1) *drafting*, where a cheaper MTP draft model generates n future tokens, and 2) *verification*, where the target STP model accepts or rejects the generated tokens in parallel according to a pre-defined consistency criterion. The closer the distributions of the draft and verifier are, the more often ‘speculated’ tokens are accepted (Leviathan et al., 2022; Sun et al., 2023), speeding up generation. With speculative decoding, we quantify the trade-off between expressiveness and efficiency in MTP models as their *throughput*:

$$\text{throughput (tok/s)} = \frac{\text{number of generated tokens}}{\text{latency (s)}}, \quad (2)$$

where the numerator is approximately the number of accepted tokens and the denominator captures the wall-clock cost of generating the tokens (see Sec. 4.2 for details). While previous work such as Basharin et al. (2025) reports accepted token counts and latency separately, we highlight how both sides of the ratio in Eq. (2) are coupled, creating a spectrum. MTPC provides a systematic way to navigate this spectrum (see Fig. 1).

3. Probabilistic Circuits for MTP

The idea behind MTPCs is to further decompose the joint distribution in Eq. (1) into a deep computational graph encoding a hierarchical mixture model, called a *probabilistic circuit* (Secs. 3.1 and 3.2), and to parameterise it with LLM embeddings (Sec. 3.3).

3.1. Probabilistic Circuits

A *circuit* (Darwiche, 2003; Choi et al., 2020; Vergari et al., 2021), c , is a parameterised computational graph⁵ over variables $\mathbf{X}$ encoding a function, $c(\mathbf{X})$, and comprises three kinds of computational units: *input*, *product*, and *sum* units. Each product or sum unit n receives the outputs of other units as inputs, denoted with the set $\text{in}(n)$. Each unit n encodes a function, c_n , defined as: (i) $c_n(\text{sc}(n); \phi)$ if n is an input unit, where c_n is a function parameterised by ϕ over variables $\text{sc}(n) \subseteq \mathbf{X}$, called its *scope*; (ii) $\prod_{j \in \text{in}(n)} c_j(\text{sc}(j))$ if n is a product unit; and (iii) $\sum_{j \in \text{in}(n)} \omega_j c_j(\text{sc}(j))$ if n is a sum unit, with $\omega_j \in \mathbb{R}$ denoting the sum parameters. The scope of a product or sum unit n is the union of the scopes of its inputs, *i.e.*, $\text{sc}(n) = \bigcup_{j \in \text{in}(n)} \text{sc}(j)$. Fig. 2 shows examples of circuits, where units of the same scope are grouped into (coloured) layers that can be easily parallelised on a GPU (Mari et al., 2023; Locante et al., 2025a).

⁵In Fig. 2, the directionality of the edges is removed for readability, but it is assumed to be from inputs to outputs.

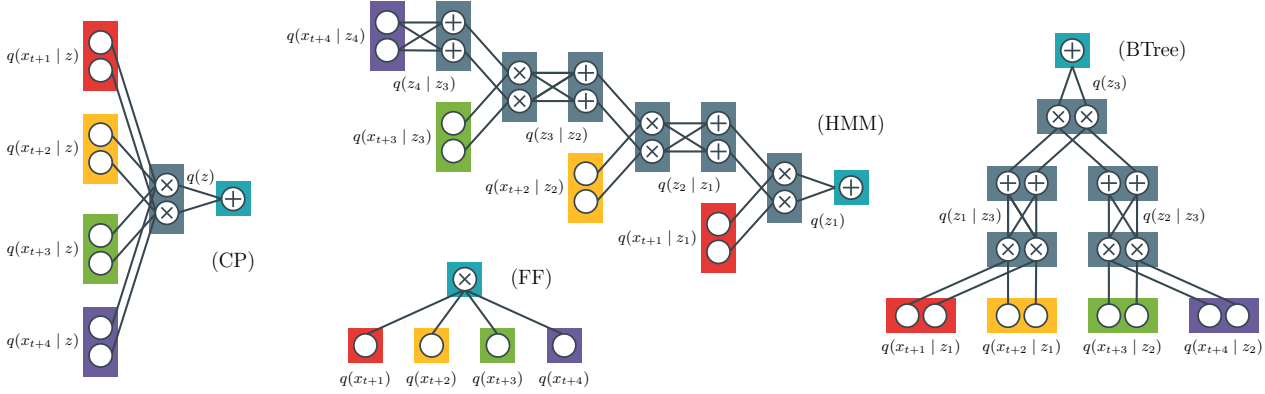


Figure 2. PCs allow for modelling a spectrum of dependency structures over sequences of tokens, as shown for the known FF and CP and the novel HMM and BTree MTP variants. Input units are grouped in coloured layers, one for each token, while sum and product layers encoding (hierarchies of) latent variable distributions are in grey. The output unit of each circuit (in blue) computes $q\theta(x_{t+1}, \dots, x_{t+n} \mid \mathbf{x}_{\leq t})$. In the figure we omit the dependency on the context $\mathbf{x}_{\leq t}$ for readability.

For MTPCs, we use *probabilistic circuits* (PCs), i.e., circuits modelling a joint distribution over random variables, in our case tokens. PCs encode Eq. (1) as

$$q\theta(x_{t+1}, \dots, x_{t+n} \mid \mathbf{x}_{\leq t}) = Z_{\theta_t}^{-1} c(x_{t+1}, \dots, x_{t+n}; \theta_t), \quad (3)$$

where $\theta_t = \{\omega_t, \phi_t\}$ denote the set of circuit parameters, i.e., all sum unit parameters ω_t and input unit parameterisations ϕ_t which depend on the context $\mathbf{x}_{\leq t}$; and Z_{θ_t} denotes the partition function of c , i.e.,

$$Z_{\theta_t} = \sum_{x_{t+1}, \dots, x_{t+n} \in \mathcal{V}^n} c(x_{t+1}, \dots, x_{t+n}; \theta_t). \quad (4)$$

Note that the PC architectures we are interested in are already normalised or always allow computing the partition function in a single feed-forward step (see Choi et al. (2020) and Sec. B.1). At the same time, we can easily sample from PCs in a single feedforward pass, as discussed in Sec. B.2. Crucially, within the framework of PCs, we can recover the FF and CP parameterisations for MTP and several other architectures that generalise tensor factorisations (Loconte et al., 2025a), each offering a different expressiveness-efficiency trade-off. We do so while abstracting away from each model’s original formulation and obtain a unified way to parameterise MTP LLMs, as discussed next.

3.2. PC Architectures for MTP

MTPC-FF. Representing the commonly used FF MTP parameterisation as a PC is simple: we introduce n input units, each parameterised by ϕ_i , its corresponding token probabilities, and connect them all to a single product unit, as shown in Fig. 2 for a distribution over $n = 4$ tokens.

MTPC-CP. Similarly, we can easily encode a CP factorisation in a *shallow* PC by i) introducing r input units for each token (each parameterised by their own probabilities

ϕ_{ij}), then ii) multiplying them to retrieve the r factorised mixture components, which we then iii) aggregate in a sum unit with weights $\omega_j = q(z_j \mid \mathbf{x}_{\leq t})$ (see also Proposition 1 in Loconte et al. (2025a)). Fig. 2 shows this construction for $n = 4$ and $r = 2$. This basic construction suggests that we can create deeper architectures by interleaving sum and product layers, while overparameterising each layer by increasing the number of units r in it. Furthermore, implementing CP as a PC unlocks a faster sampling routine (Sec. B.2) than the one used in Basharin et al. (2025).

MTPC-HMM. As a further example of the expressiveness increase we get by generalising our approach to deeper PCs, we introduce a factorisation that realises a hidden Markov model (HMM), which better captures distant dependencies in the sequence by introducing a *sequence of latent variables*, in contrast to the single one in CP. More precisely, we define an HMM with r hidden states and truncate its prediction window to n steps into the future. We resort to an inhomogeneous HMM, i.e., we use time-dependent transition matrices, as it is more expressive and worked better in our experiments (Appendix H). This simplifies Eq. (1) into:

$$\sum_{z_1=1}^r \cdots \sum_{z_n=1}^r q(z_1 \mid \mathbf{x}_{\leq t}) q\theta(x_{t+1} \mid z_1, \mathbf{x}_{\leq t}) \times \prod_{i=2}^n q(z_i \mid z_{i-1}, \mathbf{x}_{\leq t}) q\theta(x_{t+i} \mid z_i, \mathbf{x}_{\leq t}). \quad (\text{HMM})$$

Fig. 2 illustrates the HMM parameterisation above represented as a circuit, comprising $n = 4$ stacked pairs of sum and product layers, where the parameters ω_i of the former are the transition probabilities $q(z_i \mid z_{i-1}, \mathbf{x}_{\leq t})$. Similarly to CP, we can increase r to overparameterise the circuit with more input units per token and sum units overall, and hence increase expressiveness.

MTPC-BTREE. One drawback of the HMM parameterisation is the asymmetry of its computational graph, which i) provides fewer latent variables for the early tokens, and

ii) increases latency when predicting the last tokens due to its autoregressive token dependencies. To solve this, we build a PC whose structure resembles that of a binary tree (BTree), effectively encoding a *hierarchy of latent variables* or a tree tensor factorisation (Grasedyck, 2010; Cheng et al., 2019; Loconte et al., 2025a). This is done recursively: at each step h of the hierarchy, given n tokens, and a parent latent variable Z_l , we split the tokens into two sub-sequences $(x_{t+1}, \dots, x_{t+\lfloor n/2 \rfloor - 1})$ and $(x_{t+\lfloor n/2 \rfloor}, \dots, x_{t+n})$, then factorise Eq. (1) as the mixture:

$$\begin{aligned} & \sum_{z_h=1}^r q(z_h \mid z_l, \mathbf{x}_{\leq t}) \\ & \times q_{\theta}(x_{t+1}, \dots, x_{t+\lfloor n/2 \rfloor - 1} \mid z_h, z_l, \mathbf{x}_{\leq t}) \\ & \times q_{\theta}(x_{t+\lfloor n/2 \rfloor}, \dots, x_{t+n} \mid z_h, z_l, \mathbf{x}_{\leq t}) \quad (\text{BTree}) \end{aligned}$$

which corresponds to creating a sum unit whose weights are $q(z_h \mid z_l, \mathbf{x}_{\leq t})$ followed by products. We repeat the process while caching intermediate units until we reach the base case for $n = 1$, for which we create a layer of input units for the corresponding token. Fig. 2 illustrates the BTree circuit built in this way. Our experiments (Sec. 4) show that the BTree parameterisation obtains the optimal throughput by lowering the latency of the HMM, as it samples more latent variables and tokens in parallel, while achieving similar acceptance rates. In addition, our experiments show that the BTree parameterisation improves throughput on larger MTP window sizes as the balanced tree structure enables parallel sampling of tokens and latent variables.

3.3. Parameterising PCs with LLMs

Parameterising MTPCs requires two functions: an LLM that maps the context $\mathbf{x}_{\leq t} \in \mathcal{V}^t$ into contextual features, and a neural network head that maps the contextual features to the parameters of the circuit θ_t , realising a *neural conditional circuit* (Shao et al., 2020; 2022; Ahmed et al., 2022). To extract the contextual features $\mathbf{e}_t \in \mathbb{R}^d$, we use $\mathbf{e}_t = \text{LLM}_{\text{LoRA}(k)}(\mathbf{x}_{\leq t})$ where $\text{LLM}_{\text{LoRA}(k)}: \mathcal{V}^t \rightarrow \mathbb{R}^d$ is the STP backbone with LoRA (Hu et al., 2022) applied to the last $k \geq 0$ layers. As we will discuss in Sec. 4.4, the number of LoRA layers can impact throughput significantly. Given $\mathbf{e}_t$, we realise Eq. (3) by computing $\theta_t = g_c(\mathbf{e}_t)$, where g_c is a neural network head that outputs both the input unit parameters, ϕ_t , and the sum unit parameters, ω_t (Sec. 3.1). Note that our parameterisation in MTPCs allows us to abstract from the actual structure of the circuit (i.e., FF, CP, HMM or BTree) and just focus on these two sets of tensorised parameters, as we discuss next.

Input unit distributions. All MTPCs produce joint distributions over token windows by combining categorical distributions over individual tokens (Fig. 2). We follow EvaByte (Zheng et al., 2025) and learn n separate unembedding layers, one per window position. For models with mixture coefficients, we also learn one unembedding layer per

mixture coefficient.⁶ As such, instead of a single unembedding matrix mapping $\mathbb{R}^d \rightarrow \mathbb{R}^v$, we have an unembedding tensor $\mathcal{W} \in \mathbb{R}^{n \times r \times v \times d}$, and compute the input distributions with the usual unembedding operation followed by softmax, i.e., $\phi_{tij} = \text{softmax}(\mathcal{W}_{ij}\mathbf{e}_t)$, where i and j index the position in the MTP window and the rank r .

Sum unit parameters. Instead of mapping embeddings to the vocabulary via $\mathcal{W}$, we map to the rank of the sum unit via $\mathcal{R} \in \mathbb{R}^{z \times r \times d}$, where z is the number of sum units, r is its rank, and d the dimensionality of $\mathbf{e}_t$. We compute $\omega_{ti} = \text{softmax}(\mathcal{R}_i\mathbf{e}_t)$, where i indexes the sum unit.

3.4. Speculative Decoding with MTPC

For MTPCs, we design an architecture that is *self-drafting* (Zhang et al., 2024b; Cai et al., 2024), i.e. where the draft and verifier models share the same LLM backbone. We use an MTP head (Cai et al., 2024; Ankner et al., 2024) augmented with our circuits to efficiently sample a draft, and an autoregressive STP head as the verifier. Optionally, we also keep a few final transformer layers separate in the two models by fine-tuning LoRA adapters for the draft model.

Unlike previous self-drafting MTP works (Cai et al., 2024; Ankner et al., 2024), we guarantee that the generated tokens are exactly the same (*greedy speculative decoding* (Stern et al., 2018)) or the same in expectation (*speculative sampling*) as those the autoregressive LLM would generate using *speculative decoding* (Leviathan et al., 2022; Chen et al., 2023), i.e., we only generate the subset of drafted tokens accepted by our verifier. To keep latency low, we make only a single LLM call per speculative decoding cycle by re-using, where possible, the LLM backbone state computed by the verifier for the draft model. Sharing the state forfeits the bonus token of standard speculative decoding, except in the rare case where no tokens are accepted; see Alg. 2. The GPU memory overhead of speculative decoding with our most expressive PCs is only 10-15% more than MTPC-FF, see Fig. 5. Next, we report results for MTPC both for speculative sampling and greedy speculative decoding.

4. Retrofitting Byte-Level LLMs with MTPCs

We evaluate MTPC on the challenging tasks of speeding up byte-level LLMs. We speed-up two models, EvaByte 6.5B, which already has MTP capabilities, and Llama 3.2 3B (Byte), which does not and is a byte-fied version of the popular subword-level LLM. Detailed parameters of both models can be found in Appendix E. We implement our MTPCs variants in the `circuit` library (The april Lab, 2024) and provide it in our supplementary materials.

⁶This is efficient even for PCs with high rank due to the small vocabulary size of byte-level LLMs.

EvaByte 6.5B. EvaByte (Zheng et al., 2025) is an open source, publicly available byte-level model that obtains results that are competitive to subword-level LLMs on benchmarks (Zheng et al., 2025), see Sec. E.1. Importantly, EvaByte has been pretrained as an MTP model with a prediction window of $n = 8$ bytes. As a result, it can already speed-up generation very effectively with greedy speculative decoding. However, its acceptance rate with speculative sampling is hampered by the unrealistic independence assumptions which we relax with MTPC. Moreover, we will see that EvaByte’s pretraining enables us to successfully extend prediction to $n = 16$ tokens. Thus, we retrofit EvaByte-SFT (Zheng et al., 2025), which has been instruction fine-tuned on a data mix of Tülu 3 (Lambert et al., 2025), OpenCoder (Huang et al., 2025) stages one and two, and OpenHermes 2.5. We note that EvaByte’s solid performance on benchmarks is obtained via Medusa-style lossy speculative decoding with the MTP head, which in the case of sampling comes with a loss in quality compared to EvaByte-STP ($n = 1$). We therefore set EvaByte-STP as the target model for speculative decoding to *accelerate generation without sacrificing generation quality*.

Llama 3.2 3B (Byte). We also retrofit a Llama3.2 3B model (Grattafiori et al., 2024) to show that our findings generalise. Since we focus on byte-level LLMs, we retrofit the byte-level version that has been distilled from the subword-level model in Minixhofer et al. (2026) while retaining most of Llama’s downstream performance.⁷ The byte-fied Llama3.2 3B model was fine-tuned on Tülu 3 with a context length of 2048. As opposed to EvaByte, the byte-fied version of Llama was not pretrained for multi-token prediction. As such, the boost we can get from MTPC is smaller than that for EvaByte, as it is more challenging to adapt it to long windows with LoRA adapters only on the last few layers.

Draft models. Each draft model comprises a backbone and an MTP head. We introduce two axes to control the draft model’s expressiveness: i) more expressive circuit heads and ii) more LoRA layers in the draft backbone. For i) we combine the target’s model backbone with one of our MTPCs heads, including our CP implementation and novel HMM and BTree heads to relax the independence assumptions of the FF model and increase expressiveness. We note that MTPC-CP with $r = 1$ is equivalent to MTPC-FF, as can be seen from Eq. (CP). For ii) we decouple the draft and target backbone by adding LoRA adapters to the last 1, 2 or 4 layers of the draft’s backbone.

4.1. Training

We train 55 draft models in total. We improve throughput by making our MTP model’s distribution as similar as possible to the target’s in the simplest way: we instruction fine-tune

our MTP models on a similar data mix to that used for instruction fine-tuning the target.

Training data. We fine-tune on the Tülu 3 SFT mix dataset (Lambert et al., 2025) which contains 939,344 examples of user/assistant interactions on 18 tasks. We split the Tülu 3 dataset into training and validation and evaluate throughput on the unseen validation examples. We make sure all tasks are sampled by shuffling the training data before splitting. To make training possible on 2×80 GB GPUs, we limit the context length to 8192 bytes and filter out 34,067 examples which are longer. We split the remaining 905,277 examples into 99% train and 1% validation.

Initialisation. For EvaByte, we initialise our MTP heads from EvaByte-SFT in a way that guarantees that our EvaByte-MTP-CP is equivalent to EvaByte-MTP. This guarantees that we leverage previous training: all models start from the same loss and we smoothly move in parameter space from EvaByte-MTP to our more expressive EvaByte-MTP-CP, EvaByte-MTP-HMM and EvaByte-MTP-BTree.

Loss. We train our MTP models on the packed train split of Tülu 3 with a batch size of 256 sequences, or ≈ 2 m tokens, which is what EvaByte used. We first train our MTP heads for 1 epoch (Sec. 4.3). Then we load the models and continue training for an additional epoch with LoRA (Sec. 4.4). We apply the target model’s chat template and only train on the assistant’s answers. We use overlapping prediction windows, as we need to be able to begin speculative decoding from any position during generation. We minimise the negative log-likelihood of the observed assistant outputs, see Eq. (5), where N is the number of training sequences and L is the sequence length for each token in the window.⁸

$$\mathcal{L}_j = -\frac{1}{N} \sum_{i=1}^N \frac{1}{V_{i,j}} \sum_{t=1}^L \log p_{\theta}(x_{t+j}^{(i)} | \mathbf{x}_{<t+j}^{(i)}) \quad (5)$$

This involves locally normalising the loss by the number of valid tokens for example i and output j in the MTP window, $V_{i,j}$. As in Cai et al. (2024), we apply exponential discounting to the loss $\mathcal{L}_j$ of token j in the MTP window, i.e., we minimise $\mathcal{L} = \sum_{j=1}^n \gamma^{j-1} \mathcal{L}_j$.⁹ We use Adam (Kingma & Ba, 2015) with a fixed learning rate of 3×10^{-4} .

4.2. Metrics

To speed up LLM generation with speculative decoding, we need to balance **speed** and **expressiveness**. We measure speed using **mean latency per cycle** ($\odot \mu_{\text{lat}}$) and expressiveness via the **mean accepted tokens per cycle** ($\boxtimes \mu_{\text{acc}}$; Li et al., 2024), as defined below. Our goal is to increase **throughput**. We obtain a relative throughput speed-up

⁸Our loss over overlapping windows is a composite log-likelihood (Varin et al., 2011).

⁹We set $\gamma = 0.9$ instead of $\gamma = 0.8$ in the case of $n > 8$.

⁷<http://hf.co/benjamin/Llama3-2-3B-IT-Byte>

of one method over another by measuring their **wall-time speedup ratio** (Li et al., 2024; Cai et al., 2024). We use a batch size of 1 for all evaluations. Metrics for our main experiments are computed on the server-grade NVIDIA L40S GPU with complete results in Appendix K. We also run experiments on the desktop-grade NVIDIA RTX 3090 in Appendix L. We detail the metrics below.

Mean latency $\odot\mu_{\text{lat}}$ is the average time of each speculative decoding cycle, *i.e.*, the time needed for the draft model to generate a candidate sequence and the verifier to choose which tokens to accept. $\odot\mu_{\text{lat}}$ is higher for less efficient LLMs and MTP heads, and lower for more powerful GPUs.

Mean accepted tokens $\boxtimes\mu_{\text{acc}}$ is the mean number of drafted tokens that are accepted by the target model per cycle. More expressive draft models will have higher acceptance as they will better approximate the target distribution. $\boxtimes\mu_{\text{acc}}$ depends on the size of the MTP window, n , as we have $\boxtimes\mu_{\text{acc}} \in [0, n]$.

Mean throughput $\mu_{\text{tok/s}}$ is the total number of generated tokens across all prompts divided by the total generation latency, excluding prefill (Eq. (2)). In the trade-off plots, we cannot directly visualise $\mu_{\text{tok/s}}$. Instead, we decompose throughput into the ratio $\boxtimes\mu_{\text{acc}} / \odot\mu_{\text{lat}}$, which approximates Eq. (2) closely but not exactly: it is a ratio of per-cycle means rather than of totals, and accepted tokens slightly undercount generated tokens, as one token is still generated in the rare case where none are accepted (see Alg. 2).

Wall-time speed-up ratio is the relative speed-up of a proposed model compared to a baseline model, measured as the ratio of their $\mu_{\text{tok/s}}$. As baselines, we use autoregressive generation from the STP target model and MTP with independence assumptions.

4.3. MTPCs without Adapters

RQ1: Can we increase throughput by increasing the number of mixture components?

We begin with the simplest PC from our framework, MTPC-CP, which relaxes the independence assumption of the widely used MTPC-FF ($r = 1$) by increasing the number of mixture coefficients, r . MTPC-CP *increases throughput as it is more expressive than MTPC-FF, yet still very efficient*.

Table 1 highlights MTPC-CP’s increase in expressivity as evidenced by the increase in $\boxtimes\mu_{\text{acc}}$ as we increase r : MTPC-CP reaches $\boxtimes\mu_{\text{acc}} = 6.02$ for $r = 128$, surpassing the MTPC-FF baseline by .83 for EvaByte with speculative sampling. At the same time, the $\odot\mu_{\text{lat}}$ introduced by MTPC-CP increases only slightly as we increase r , because the forward pass cost of the MTPC-CP head is dominated by the expensive LLM calls. Nevertheless, as the increase in $\boxtimes\mu_{\text{acc}}$ tails off for larger r , even the small $\odot\mu_{\text{lat}}$ increase matters and we see the first of three cases in our experiments where the

Table 1. Increasing the mixture components (r) increases the throughput ($\mu_{\text{tok/s}}$) as seen for MTPC-CP ($n = 8$) over our baseline, EvaByte-MTP (FF) (in gray) where we report the mean $\pm$ std over three sets of 250 prompts of Tulu3 on an L40S GPU. MTPC-CP increases throughput: it has a larger acceptance ($\boxtimes\mu_{\text{acc}}$) while latency ($\odot\mu_{\text{lat}}$) increases slowly in r .

		$\odot\mu_{\text{lat}} \downarrow$	$\boxtimes\mu_{\text{acc}} \uparrow$	$\mu_{\text{tok/s}} \uparrow$	$\text{max}_{\text{tok/s}}$
FF	1	0.0299 ± 0.0003	5.19 ± 0.05	176.3 ± 0.9	297.55
	8	0.0314 ± 0.0005	5.67 ± 0.05	184.2 ± 3.9	297.92
	16	0.0323 ± 0.0020	5.79 ± 0.08	183.4 ± 13.3	290.71
CP	32	0.0314 ± 0.0003	5.88 ± 0.02	191.0 ± 1.6	287.88
	64	0.0327 ± 0.0015	5.96 ± 0.04	185.9 ± 7.7	281.10
	128	0.0337 ± 0.0001	6.02 ± 0.03	182.3 ± 1.1	264.80

expressiveness-latency trade-off is vital for understanding the outcome: the best throughput is obtained for $r = 32$ and not $r = 128$. We note that our findings for MTPC-CP also hold for Llama and greedy speculative decoding, see Tables 10, 13 and 16 in the Appendix. In the last column, we also show the maximum attainable throughput ($\text{max}_{\text{tok/s}}$), *i.e.*, we disable speculative decoding and accept all tokens. We see that we pay ≈ 90 tok/s for $r = 32$, which is not a big price to pay for guaranteeing no loss in generation quality.

While MTPC-CP performs well for $n = 8$, the margin for further improving throughput is small. This is because for $n = 8$, we can at best achieve $\boxtimes\mu_{\text{acc}} = 8$, and we have already achieved $\boxtimes\mu_{\text{acc}} = 6.02$ and have hit diminishing returns. To obtain substantial boosts in throughput, we need to find another axis of expressivity to benefit from. We therefore use MTPC’s more expressive circuits to extend our model to longer window sizes. As $r = 32$ worked best, we keep it fixed for the remaining experiments.

RQ2: Do we benefit from more expressive circuit architectures for longer MTP windows?

We now consider the more expressive MTPC-HMM and MTPC-BTREE, and show that they outperform MTPC-CP on $\boxtimes\mu_{\text{acc}}$ for longer MTP windows, highlighting the importance of our extension to general PCs. We fix $r = 32$ and explore the different PC architectures for both $n = 8$ and the longer window, $n = 16$. Fig. 3 shows that both MTPC-HMM and MTPC-BTREE increase $\boxtimes\mu_{\text{acc}}$. However, MTPC-HMM strikes an unfavourable balance in the expressiveness-latency trade-off: *Due to being AR, MTPC-HMM has the largest $\odot\mu_{\text{lat}}$ (see Tables 8, 11, 14 and 17), and yields poor throughput as a result*.

Interestingly, for greedy decoding EvaByte-MTP achieves higher throughput than more expressive circuits (see Fig. 3, bottom right). We conjecture this is because EvaByte was pretrained with the independence assumption, so its representations are well-aligned with a factorised head such as

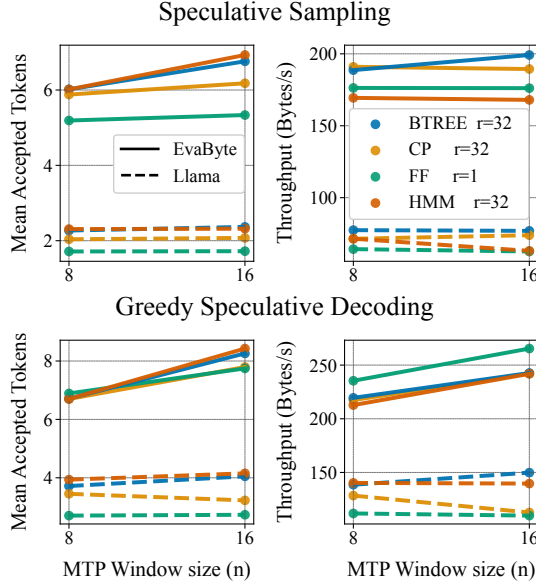


Figure 3. Extending the MTP window of expressive PCs such as MTPC-HMM and MTPC-BTREE to $n = 16$ boosts their μ_{acc} in all settings, as can be seen from the upward trend on the left subplots and in more detail in Fig. 6. Importantly, MTPC-BTREE improves $\mu_{tok/s}$ for all but Llama with speculative sampling, highlighting its strong expressiveness–latency trade-off. In contrast, MTPC-HMM lags behind due to high latency from its AR nature.

MTPC-FF. This suggests that expressive circuits are not always necessary for greedy decoding, which requires only the correct argmax, but appear more beneficial for sampling, which requires approximating the full distribution.

While the gains already obtained by MTPC-BTREE are solid, fine-tuning the MTP head alone can only get us so far. This is because neither Llama nor EvaByte has been trained to produce representations that are good for predicting 16 tokens ahead, as we discuss next.

TAKEAWAY 1: While increasing the mixture components r in CP is initially beneficial, it soon hits diminishing returns. Increasing the MTP future token window size n and adopting more expressive PC architectures unlocks further throughput gains. Moreover, while the HMM achieves the highest acceptance rates, it incurs high latency. Instead, *non-autoregressive* variants such as BTREE strike a better trade-off and should be preferred.

4.4. MTPCs with Adapters

RQ3: Can we further increase throughput by adapting the draft LLM using LoRA?

We now consider our final axis for increasing expressiveness: adding LoRA layers to the draft model. We note that we also fine-tune a model with 0 LoRA layers for an additional epoch to highlight that our improvements are not

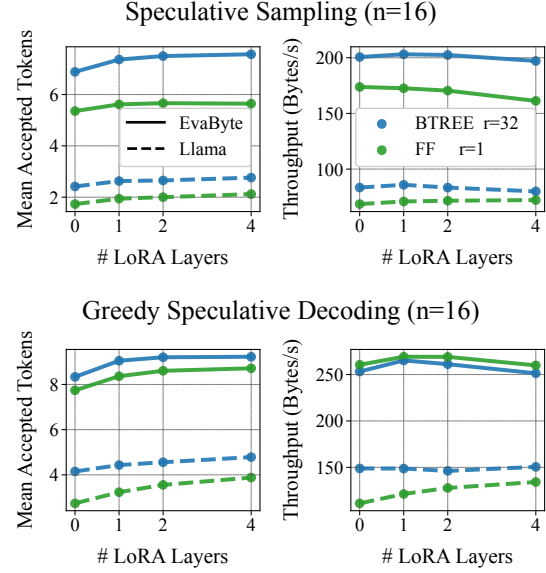


Figure 4. We increase the expressiveness of the draft model by adding LoRA layers (x-axis) for $n = 16$. As a result, the μ_{acc} (left subplots) increases substantially for both EvaByte and Llama for both speculative decoding settings. Due to the expressiveness–latency trade-off the optimal $\mu_{tok/s}$ for EvaByte is obtained with 1 LoRA layer while Llama often benefits further from 2 or more.

due to longer training alone. We show that while we can improve throughput, we need to be strategic when choosing the number of LoRA layers, as the latency introduced can rapidly outweigh the expressiveness gained.

For example, if we train adapters for the last 16 (out of 32) layers of EvaByte, we can improve μ_{acc} by 37%, but we introduce a $\mathcal{O}(\mu_{lat})$ of $1.5\times$ the cost of a forward pass of the LLM.¹⁰ As can be seen for $n = 8$ in Fig. 4, adding LoRA layers almost always improves throughput. An exception is EvaByte-MTP with $n = 8$ where the μ_{acc} cannot be increased by adding LoRA layers, see Fig. 7. This is likely because the backbone has been pretrained with the MTP loss, and its representations cannot be improved further. In contrast, EvaByte-MTP-BTree once again obtains larger increases in μ_{acc} , especially for speculative sampling for $n = 16$ where by using 4 LoRA layers we obtain a boost of .69 in μ_{acc} over having no LoRA layers. However, because of the latency introduced, the best throughput is obtained with EvaByte-MTP-BTree with 1 LoRA layer, having a speed-up ($\times_{STP}$) of $\times 5.15$ over EvaByte-STP (see Table 9), while Llama benefits from 2 LoRA layers and obtains $\times 2.24$ speed-up over Llama-STP (see Table 15). Importantly, both obtain a $\times 1.17$ speed-up over the best FF MTP model, highlighting the importance of relaxing the independence assumptions to improve speculative sampling.

¹⁰We found that training more than 16 layers of EvaByte does not lead to improvements in acceptance rates.

TAKEAWAY 2: Fine-tuning a few layers of the draft model with LoRA increases the number of accepted tokens but also increases latency. The optimal trade-off can be device and backbone specific, but adding LoRAs is always beneficial compared to a fully shared LLM backbone, as long as we are retrofitting a model to an MTP window size it has not already been pretrained for.

5. Conclusions

We have identified key trade-offs between acceptance rates and latency within our framework, MTPC. We enhanced the *expressiveness* of MTP: we relaxed the independence assumption (Cai et al., 2024; Zheng et al., 2025) by introducing an explicit probabilistic model for inter-token dependencies, and generalised mixture-based methods (Basharin et al., 2025) into the PC framework, unlocking a better expressiveness-latency trade-off. We also showed how to further optimise the trade-off by modulating the number of layers shared between draft and verifier model backbones.

We showcased the throughput gains of MTPC LLMs *at scale* by retrofitting EvaByte (Zheng et al., 2025), a cutting-edge 6.5B byte-level LLM, and Llama 3.2 3B (Minixhofer et al., 2025), a byte-fied version of the widely used subword-level LLM, into our framework. Overall, our results show that throughput in MTP byte-level LLMs can be increased by $1.17\times$ over MTP with independence assumptions and $5.15\times/2.24\times$ over AR for EvaByte/Llama. Crucially, these speed-ups are obtained without sacrificing generation quality: we guarantee to match the target AR LLM exactly for greedy decoding (Stern et al., 2018) and in expectation for sampling (Leviathan et al., 2022).

In future work, our framework can be extended by integrating constraints during generation (Ahmed et al., 2025) or speculative decoding (Nakshatri et al., 2025) via methods such as Gelato (Zhang et al., 2023), Ctrl-G (Zhang et al., 2024a) and LTLA (Yidou-Weng et al., 2026). Unlike these methods, we would not need to train an auxiliary HMM in MTPC and could integrate constraints directly into our PC head. Moreover, we can further boost expressiveness by leveraging other PC architectures such as subtractive mixtures (Loconte et al., 2024; 2025b) and continuous latent variable circuits (Gala et al., 2024a;b), while reducing latency via recent advancements in scaling up PCs (Liu et al., 2024; Zhang et al., 2025).

Limitations

Batch Size. We do not analyse the effect of using a batch size greater than one as we want to isolate our throughput gains to the expressiveness/latency trade-off. **Languages.** Byte-level LLMs are well-suited for multilingual generation

almost out-of-the-box as they can represent any word/script as a sequence of UTF-8 bytes. However, we only consider generation in English, as we found that EvaByte tended to switch between languages during generation and showed a large variance in throughput for different languages. While exploring the effect of multi-lingual generation is an interesting avenue for future work, we believe further pretraining of EvaByte or using other byte-level LLMs would be needed. **EOS Token.** In our experiments we restrict our models from producing the end-of-sequence token, as we want to avoid situations where some models obtain better throughput by generating shorter sequences. **MTPC on Subword-level LLMs.** While MTPC can be applied to subword-level LLMs with vocabulary sizes in the tens to hundreds of thousands, the amount of GPU memory needed during training to store the logits scales linearly in the vocabulary size as $s \times n \times v \times r$, where s is the context length, n the MTP window length, v the vocabulary size and r the number of mixture components. In the paper we use a context length $s = 8192$ for byte-level LLMs and we explored $r \in \{8, 16, 32, 64, 128\}$. However, for subword-level LLMs with a vocabulary size that is $100\times$ larger, we would hit a memory bottleneck with anything more than $r = 4$ on a GPU with the same memory (80 GB), as we show in Table 5 in Appendix G. We discuss approaches for mitigating this memory bottleneck in Appendix G. These are promising for extending MTPC to subword-level LLMs, but require additional ablations or non-trivial adaptations that were out of scope for this paper.

Acknowledgements

We would like to thank the anonymous reviewers for their thorough feedback and suggestions. We also thank Lin Zheng, author of EvaByte, for help with the EvaByte codebase and for answering all of our questions on EvaByte. Moreover, we thank Benjamin Minixhofer for making their models available and for answering questions about the implementation. We would also like to thank the members of april lab and Ponti lab for useful feedback during early presentations of this work.

AG was funded by NatWest Group and ERC grant (“Numerical Analysis for Stable AI”, 101198795). EvK was funded by the ELIAI project “Gradient-based Learning of Complex Latent Structures” and the NWO AiNed project “Human-Centric AI Agents with Common Sense” (NGF.1607.22.044). EP is supported by the ERC Starting Grant ATOM-FM (101222956) “Adaptive Memory and Tokenization in Foundation Models for Efficient and Long-Horizon AI”. AG and AV were funded by the “UNREAL: Unified Reasoning Layer for Trustworthy ML” project (EP/Y023838/1) selected by the ERC and funded by UKRI EPSRC.

Impact statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

Reproducibility statement

To ensure reproducibility for our research, we have released the code and checkpoints for all model variants at github.com/april-tools/mtpc. In addition, we have provided the full details of sampling in circuits in Appendix B and of our algorithms for speculative decoding in Appendix F.

References

- Ahia, O., Kumar, S., Gonen, H., Kasai, J., Mortensen, D., Smith, N., and Tsvetkov, Y. Do all languages cost the same? Tokenization in the era of commercial language models. In Bouamor, H., Pino, J., and Bali, K. (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 9904–9923, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.614. URL <https://aclanthology.org/2023.emnlp-main.614>.
- Ahmed, K., Teso, S., Chang, K.-W., Van den Broeck, G., and Vergari, A. Semantic probabilistic layers for neuro-symbolic learning. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, NIPS ’22, Red Hook, NY, USA, 2022. Curran Associates Inc. ISBN 9781713871088.
- Ahmed, K., Chang, K.-W., and Van den Broeck, G. Controllable generation via locally constrained resampling. In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 4 2025. URL <https://arxiv.org/pdf/2410.13111>.
- Ankner, Z., Parthasarathy, R., Nrusimha, A., Rinard, C., Ragan-Kelley, J., and Brandon, W. Hydra: Sequentially-dependent draft heads for medusa decoding. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=FbhjirzvJG>.
- Basharin, A., Chertkov, A., and Oseledets, I. Faster language models with better multi-token prediction using tensor decomposition, 2025. URL <https://arxiv.org/abs/2410.17765>.
- Cai, T., Li, Y., Geng, Z., Peng, H., Lee, J. D., Chen, D., and Dao, T. Medusa: Simple llm inference acceleration framework with multiple decoding heads. In *Proceedings of the 41st International Conference on Machine Learning*, ICML’24. JMLR.org, 2024.
- Chen, C., Borgeaud, S., Irving, G., Lespiau, J.-B., Sifre, L., and Jumper, J. Accelerating large language model decoding with speculative sampling, 2023. URL <https://arxiv.org/abs/2302.01318>.
- Cheng, S., Wang, L., Xiang, T., and Zhang, P. Tree tensor networks for generative modeling. *Physical Review B*, 99 (15):155131, 2019.
- Choi, Y., Vergari, A., and Van den Broeck, G. Probabilistic circuits: A unifying framework for tractable probabilistic modeling. Technical report, University of California, Los Angeles (UCLA), 2020. URL <http://starai.cs.ucla.edu/papers/ProbCirc20.pdf>.
- Dagan, G., Synnaeve, G., and Rozière, B. Getting the most out of your tokenizer for pre-training and domain adaptation. In *Proceedings of the 41st International Conference on Machine Learning*, ICML’24. JMLR.org, 2024.
- Darwiche, A. A differential approach to inference in bayesian networks. *Journal of the ACM (JACM)*, 50: 280–305, 2003.
- Darwiche, A. *Modeling and Reasoning with Bayesian Networks*. Cambridge University Press, 2009.
- Darwiche, A. and Marquis, P. A knowledge compilation map. *Journal of Artificial Intelligence Research (JAIR)*, 17:229–264, 2002.
- DeepSeek-AI, Liu, A., Feng, B., Xue, B., Wang, B., Wu, B., Lu, C., Zhao, C., Deng, C., Zhang, C., Ruan, C., Dai, D., Guo, D., Yang, D., Chen, D., Ji, D., Li, E., Lin, F., Dai, F., Luo, F., Hao, G., Chen, G., Li, G., Zhang, H., Bao, H., Xu, H., Wang, H., Zhang, H., Ding, H., Xin, H., Gao, H., Li, H., Qu, H., Cai, J. L., Liang, J., Guo, J., Ni, J., Li, J., Wang, J., Chen, J., Chen, J., Yuan, J., Qiu, J., Li, J., Song, J., Dong, K., Hu, K., Gao, K., Guan, K., Huang, K., Yu, K., Wang, L., Zhang, L., Xu, L., Xia, L., Zhao, L., Wang, L., Zhang, L., Li, M., Wang, M., Zhang, M., Zhang, M., Tang, M., Li, M., Tian, N., Huang, P., Wang, P., Zhang, P., Wang, Q., Zhu, Q., Chen, Q., Du, Q., Chen, R. J., Jin, R. L., Ge, R., Zhang, R., Pan, R., Wang, R., Xu, R., Zhang, R., Chen, R., Li, S. S., Lu, S., Zhou, S., Chen, S., Wu, S., Ye, S., Ye, S., Ma, S., Wang, S., Zhou, S., Yu, S., Zhou, S., Pan, S., Wang, T., Yun, T., Pei, T., Sun, T., Xiao, W. L., Zeng, W., Zhao, W., An, W., Liu, W., Liang, W., Gao, W., Yu, W., Zhang, W., Li, X. Q., Jin, X., Wang, X., Bi, X., Liu, X., Wang, X., Shen, X., Chen, X., Zhang, X., Chen, X., Nie, X., Sun, X., Wang, X., Cheng, X., Liu, X., Xie, X., Liu, X., Yu, X., Song, X., Shan, X., Zhou, X., Yang, X., Li, X., Su, X., Lin, X.,

- Li, Y. K., Wang, Y. Q., Wei, Y. X., Zhu, Y. X., Zhang, Y., Xu, Y., Xu, Y., Huang, Y., Li, Y., Zhao, Y., Sun, Y., Li, Y., Wang, Y., Yu, Y., Zheng, Y., Zhang, Y., Shi, Y., Xiong, Y., He, Y., Tang, Y., Piao, Y., Wang, Y., Tan, Y., Ma, Y., Liu, Y., Guo, Y., Wu, Y., Ou, Y., Zhu, Y., Wang, Y., Gong, Y., Zou, Y., He, Y., Zha, Y., Xiong, Y., Ma, Y., Yan, Y., Luo, Y., You, Y., Liu, Y., Zhou, Y., Wu, Z. F., Ren, Z. Z., Ren, Z., Sha, Z., Fu, Z., Xu, Z., Huang, Z., Zhang, Z., Xie, Z., Zhang, Z., Hao, Z., Gou, Z., Ma, Z., Yan, Z., Shao, Z., Xu, Z., Wu, Z., Zhang, Z., Li, Z., Gu, Z., Zhu, Z., Liu, Z., Li, Z., Xie, Z., Song, Z., Gao, Z., and Pan, Z. Deepseek-v3 technical report, 2024. URL <https://arxiv.org/abs/2412.19437>.
- Gala, G., de Campos, C., Peharz, R., Vergari, A., and Quaeghebeur, E. Probabilistic integral circuits. In *AIS-TATS 2024*, 2024a.
- Gala, G., de Campos, C., Vergari, A., and Quaeghebeur, E. Scaling continuous latent variable models as probabilistic integral circuits. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024b. URL <https://openreview.net/forum?id=Ke40kfOT2E>.
- Geiping, J., Stein, A., Shu, M., Saifullah, K., Wen, Y., and Goldstein, T. Coercing LLMs to do and reveal (almost) anything. In *ICLR 2024 Workshop on Secure and Trustworthy Large Language Models*, 2024. URL <https://openreview.net/forum?id=Y5inHAjMu0>.
- Gloeckle, F., Idrissi, B. Y., Rozière, B., Lopez-Paz, D., and Synnaeve, G. Better & faster large language models via multi-token prediction. In *Proceedings of the 41st International Conference on Machine Learning, ICML’24*. JMLR.org, 2024.
- Grasedyck, L. Hierarchical singular value decomposition of tensors. *SIAM journal on matrix analysis and applications*, 31(4):2029–2054, 2010.
- Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., Yang, A., Fan, A., Goyal, A., Hartshorn, A., Yang, A., Mitra, A., Sravankumar, A., Korenev, A., Hinsvark, A., Rao, A., Zhang, A., Rodriguez, A., Gregerson, A., Spataru, A., Roziere, B., Biron, B., Tang, B., Chern, B., Caucheteux, C., Nayak, C., Bi, C., Marra, C., McConnell, C., Keller, C., Touret, C., Wu, C., Wong, C., Ferrer, C. C., Nikolaidis, C., Allonsius, D., Song, D., Pintz, D., Livshits, D., Wyatt, D., Esiobu, D., Choudhary, D., Mahajan, D., Garcia-Olano, D., Perino, D., Hupkes, D., Lakomkin, E., AlBadawy, E., Lobanova, E., Dinan, E., Smith, E. M., Radenovic, F., Guzmán, F., Zhang, F., Synnaeve, G., Lee, G., Anderson, G. L., Thattai, G., Nail, G., Mialon, G., Pang, G., Cucurell, G., Nguyen, H., Korevaar, H., Xu, H., Touvron, H., Zarov, I., Ibarra, I. A., Kloumann, I., Misra, I., Evtimov, I., Zhang, J., Copet, J., Lee, J., Geffert, J., Vranes, J., Park, J., Mahadeokar, J., Shah, J., van der Linde, J., Billock, J., Hong, J., Lee, J., Fu, J., Chi, J., Huang, J., Liu, J., Wang, J., Yu, J., Bitton, J., Spisak, J., Park, J., Rocca, J., Johnstun, J., Saxe, J., Jia, J., Alwala, K. V., Prasad, K., Upasani, K., Plawiak, K., Li, K., Heafield, K., Stone, K., El-Arini, K., Iyer, K., Malik, K., Chiu, K., Bhalla, K., Lakhota, K., Rantala-Yearly, L., van der Maaten, L., Chen, L., Tan, L., Jenkins, L., Martin, L., Madaan, L., Malo, L., Blecher, L., Landzaat, L., de Oliveira, L., Muzzi, M., Pasupuleti, M., Singh, M., Paluri, M., Kardas, M., Tsimpoukelli, M., Oldham, M., Rita, M., Pavlova, M., Kambadur, M., Lewis, M., Si, M., Singh, M. K., Hassan, M., Goyal, N., Torabi, N., Bashlykov, N., Bogoychev, N., Chatterji, N., Zhang, N., Duchenne, O., Çelebi, O., Alrassy, P., Zhang, P., Li, P., Vasic, P., Weng, P., Bhargava, P., Dubal, P., Krishnan, P., Koura, P. S., Xu, P., He, Q., Dong, Q., Srinivasan, R., Ganapathy, R., Calderer, R., Cabral, R. S., Stojnic, R., Raileanu, R., Maheswari, R., Girdhar, R., Patel, R., Sauvestre, R., Polidoro, R., Sumbaly, R., Taylor, R., Silva, R., Hou, R., Wang, R., Hosseini, S., Chennabasappa, S., Singh, S., Bell, S., Kim, S. S., Edunov, S., Nie, S., Narang, S., Raparthy, S., Shen, S., Wan, S., Bhosale, S., Zhang, S., Vandenhende, S., Batra, S., Whitman, S., Sootla, S., Collot, S., Gururangan, S., Borodinsky, S., Herman, T., Fowler, T., Sheasha, T., Georgiou, T., Scialom, T., Speckbacher, T., Mihaylov, T., Xiao, T., Karn, U., Goswami, V., Gupta, V., Ramanathan, V., Kerkez, V., Conguet, V., Do, V., Vogeti, V., Albiero, V., Petrovic, V., Chu, W., Xiong, W., Fu, W., Meers, W., Martinet, X., Wang, X., Wang, X., Tan, X. E., Xia, X., Xie, X., Jia, X., Wang, X., Goldschlag, Y., Gaur, Y., Babaei, Y., Wen, Y., Song, Y., Zhang, Y., Li, Y., Mao, Y., Coudert, Z. D., Yan, Z., Chen, Z., Papakipos, Z., Singh, A., Srivastava, A., Jain, A., Kelsey, A., Shajnfeld, A., Gangidi, A., Victoria, A., Goldstand, A., Menon, A., Sharma, A., Boesenberg, A., Baevski, A., Feinstein, A., Kallet, A., Sangani, A., Teo, A., Yunus, A., Lupu, A., Alvarado, A., Caples, A., Gu, A., Ho, A., Poulton, A., Ryan, A., Ramchandani, A., Dong, A., Franco, A., Goyal, A., Saraf, A., Chowdhury, A., Gabriel, A., Bharambe, A., Eisenman, A., Yazdan, A., James, B., Maurer, B., Leonhardi, B., Huang, B., Loyd, B., Paola, B. D., Paranjape, B., Liu, B., Wu, B., Ni, B., Hancock, B., Wasti, B., Spence, B., Stojkovic, B., Gamido, B., Montalvo, B., Parker, C., Burton, C., Mejia, C., Liu, C., Wang, C., Kim, C., Zhou, C., Hu, C., Chu, C.-H., Cai, C., Tindal, C., Feichtenhofer, C., Gao, C., Civin, D., Beaty, D., Kreymer, D., Li, D., Adkins, D., Xu, D., Testuggine, D., David, D., Parikh, D., Liskovich, D., Foss, D., Wang, D., Le, D., Holland, D., Dowling, E., Jamil, E., Montgomery, E., Presani, E., Hahn, E., Wood, E., Le, E.-T., Brinkman, E., Arcaute, E., Dunbar, E., Smothers, E., Sun, F., Kreuk, F., Tian, F., Kokkinos, F., Ozgenel, F., Cag-

- gioni, F., Kanayet, F., Seide, F., Florez, G. M., Schwarz, G., Badeer, G., Swee, G., Halpern, G., Herman, G., Sizov, G., Guangyi, Zhang, Lakshminarayanan, G., Inan, H., Shojanazeri, H., Zou, H., Wang, H., Zha, H., Habeeb, H., Rudolph, H., Suk, H., Aspegren, H., Goldman, H., Zhan, H., Damla, I., Molybog, I., Tufanov, I., Leontiadis, I., Veliche, I.-E., Gat, I., Weissman, J., Geboski, J., Kohli, J., Lam, J., Asher, J., Gaya, J.-B., Marcus, J., Tang, J., Chan, J., Zhen, J., Reizenstein, J., Teboul, J., Zhong, J., Jin, J., Yang, J., Cummings, J., Carvill, J., Shepard, J., McPhee, J., Torres, J., Ginsburg, J., Wang, J., Wu, K., U, K. H., Saxena, K., Khandelwal, K., Zand, K., Matosich, K., Veeraraghavan, K., Michelena, K., Li, K., Jagadeesh, K., Huang, K., Chawla, K., Huang, K., Chen, L., Garg, L., A, L., Silva, L., Bell, L., Zhang, L., Guo, L., Yu, L., Moshkovich, L., Wehrstedt, L., Khabsa, M., Avalani, M., Bhatt, M., Mankus, M., Hasson, M., Lennie, M., Reso, M., Groshev, M., Naumov, M., Lathi, M., Keneally, M., Liu, M., Seltzer, M. L., Valko, M., Restrepo, M., Patel, M., Vyatskov, M., Samvelyan, M., Clark, M., Macey, M., Wang, M., Hermoso, M. J., Metanat, M., Rastegari, M., Bansal, M., Santhanam, N., Parks, N., White, N., Bawa, N., Singhal, N., Egebo, N., Usunier, N., Mehta, N., Laptev, N. P., Dong, N., Cheng, N., Chernoguz, O., Hart, O., Salpekar, O., Kalinli, O., Kent, P., Parekh, P., Saab, P., Balaji, P., Rittner, P., Bontrager, P., Roux, P., Dollar, P., Zvyagina, P., Ratanchandani, P., Yuvraj, P., Liang, Q., Alao, R., Rodriguez, R., Ayub, R., Murthy, R., Nayani, R., Mitra, R., Parthasarathy, R., Li, R., Hogan, R., Battey, R., Wang, R., Howes, R., Rinott, R., Mehta, S., Siby, S., Bondu, S. J., Datta, S., Chugh, S., Hunt, S., Dhillon, S., Sidorov, S., Pan, S., Mahajan, S., Verma, S., Yamamoto, S., Ramaswamy, S., Lindsay, S., Lindsay, S., Feng, S., Lin, S., Zha, S. C., Patil, S., Shankar, S., Zhang, S., Zhang, S., Wang, S., Agarwal, S., Sajuyigbe, S., Chintala, S., Max, S., Chen, S., Kehoe, S., Satterfield, S., Govindaprasad, S., Gupta, S., Deng, S., Cho, S., Virk, S., Subramanian, S., Choudhury, S., Goldman, S., Remez, T., Glaser, T., Best, T., Koehler, T., Robinson, T., Li, T., Zhang, T., Matthews, T., Chou, T., Shaked, T., Vontimitta, V., Ajayi, V., Montanez, V., Mohan, V., Kumar, V. S., Mangla, V., Ionescu, V., Poenaru, V., Mihailescu, V. T., Ivanov, V., Li, W., Wang, W., Jiang, W., Bouaziz, W., Constable, W., Tang, X., Wu, X., Wang, X., Wu, X., Gao, X., Kleinman, Y., Chen, Y., Hu, Y., Jia, Y., Qi, Y., Li, Y., Zhang, Y., Zhang, Y., Adi, Y., Nam, Y., Yu, Wang, Zhao, Y., Hao, Y., Qian, Y., Li, Y., He, Y., Rait, Z., DeVito, Z., Rosnbrick, Z., Wen, Z., Yang, Z., Zhao, Z., and Ma, Z. The llama 3 herd of models, 2024. URL <https://arxiv.org/abs/2407.21783>.
- Hu, E. J., yelong shen, Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=nZeVKeeFYf9>.
- Huang, S., Cheng, T., Liu, J. K., Xu, W., Hao, J., Song, L., Xu, Y., Yang, J., Liu, J., Zhang, C., Chai, L., Yuan, R., Luo, X., Wang, Q., Fan, Y., Zhu, Q., Zhang, Z., Gao, Y., Fu, J., Liu, Q., Li, H., Zhang, G., Qi, Y., Yinghui, X., Chu, W., and Wang, Z. OpenCoder: The open cookbook for top-tier code large language models. In Che, W., Nabende, J., Shutova, E., and Pilehvar, M. T. (eds.), *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 33167–33193, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.1591. URL <https://aclanthology.org/2025.acl-long.1591/>.
- Kim, Y. and Rush, A. M. Sequence-level knowledge distillation. In Su, J., Duh, K., and Carreras, X. (eds.), *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pp. 1317–1327, Austin, Texas, November 2016. Association for Computational Linguistics. doi: 10.18653/v1/D16-1139. URL <https://aclanthology.org/D16-1139/>.
- Kingma, D. P. and Ba, J. Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations (ICLR)*, 2015.
- Kolda, T. G. and Bader, B. W. Tensor decompositions and applications. *SIAM review*, 51(3):455–500, 2009.
- Lambert, N., Morrison, J., Pyatkin, V., Huang, S., Ivison, H., Brahman, F., Miranda, L. J. V., Liu, A., Dziri, N., Lyu, X., Gu, Y., Malik, S., Graf, V., Hwang, J. D., Yang, J., Bras, R. L., Tafjord, O., Wilhelm, C., Soldaini, L., Smith, N. A., Wang, Y., Dasigi, P., and Hajishirzi, H. Tulu 3: Pushing frontiers in open language model post-training. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=ilUGbfHHpH>.
- Łańcucki, A., Staniszewski, K., Nawrot, P., and Ponti, E. Inference-time hyper-scaling with KV cache compression. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. URL <https://openreview.net/forum?id=8ZiElzQxfl>.
- Land, S. and Bartolo, M. Fishing for magikarp: Automatically detecting under-trained tokens in large language models. In Al-Onaizan, Y., Bansal, M., and Chen, Y.-N. (eds.), *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 11631–11646, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi:

- 10.18653/v1/2024.emnlp-main.649. URL <https://aclanthology.org/2024.emnlp-main.649/>.
- Lee, J., Mansimov, E., and Cho, K. Deterministic non-autoregressive neural sequence modeling by iterative refinement. In Riloff, E., Chiang, D., Hockenmaier, J., and Tsujii, J. (eds.), *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pp. 1173–1182, Brussels, Belgium, October–November 2018. Association for Computational Linguistics. doi: 10.18653/v1/D18-1149. URL <https://aclanthology.org/D18-1149/>.
- Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning (ICML)*, 2022.
- Li, Y., Wei, F., Zhang, C., and Zhang, H. Eagle: speculative sampling requires rethinking feature uncertainty. In *Proceedings of the 41st International Conference on Machine Learning, ICML’24*. JMLR.org, 2024.
- Liu, A., Ahmed, K., and Van Den Broeck, G. Scaling tractable probabilistic circuits: a systems perspective. In *Proceedings of the 41st International Conference on Machine Learning, ICML’24*. JMLR.org, 2024.
- Loconte, L., Aleksanteri, M. S., Mengel, S., Trapp, M., Solin, A., Gillis, N., and Vergari, A. Subtractive mixture models via squaring: Representation and learning. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- Loconte, L., Mari, A., Gala, G., Peharz, R., de Campos, C., Quaeghebeur, E., Vessio, G., and Vergari, A. What is the Relationship between Tensor Factorizations and Circuits (and How Can We Exploit it)? *Transactions on Machine Learning Research*, 2025a. ISSN 2835-8856. Featured Certification.
- Loconte, L., Mengel, S., and Vergari, A. Sum of Squares Circuits. In *The 39th Annual AAAI Conference on Artificial Intelligence (AAAI)*, 2025b.
- Mari, A., Vessio, G., and Vergari, A. Unifying and understanding overparameterized circuit representations via low-rank tensor decompositions. In *6th Workshop on Tractable Probabilistic Modeling*, 2023.
- Minixhofer, B., Murray, T., Limisiewicz, T., Korhonen, A., Zettlemoyer, L., Smith, N. A., Ponti, E. M., Soldaini, L., and Hofmann, V. Bolmo: Byteifying the next generation of language models, 2025. URL <https://arxiv.org/abs/2512.15586>.
- Minixhofer, B., Vulić, I., and Ponti, E. Universal cross-tokenizer distillation via approximate likelihood matching. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. URL <https://openreview.net/forum?id=DxKP2E0xK2>.
- Mnih, A. and Hinton, G. E. A scalable hierarchical distributed language model. In Koller, D., Schuurmans, D., Bengio, Y., and Bottou, L. (eds.), *Advances in Neural Information Processing Systems*, volume 21. Curran Associates, Inc., 2008. URL https://proceedings.neurips.cc/paper_files/paper/2008/file/1e056d2b0ebd5c878c550da6ac5d3724-Paper.pdf.
- Nakshatri, N. S., Roy, S., Das, R., Chaidaroon, S., Boytsov, L., and Gangadharaiyah, R. Constrained decoding with speculative lookaheads. In Chiruzzo, L., Ritter, A., and Wang, L. (eds.), *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 4681–4700, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics. ISBN 979-8-89176-189-6. doi: 10.18653/v1/2025.naacl-long.239. URL <https://aclanthology.org/2025.naacl-long.239/>.
- Nawrot, P., Chorowski, J., Lancucki, A., and Ponti, E. M. Efficient transformers with dynamic token pooling. In Rogers, A., Boyd-Graber, J., and Okazaki, N. (eds.), *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 6403–6417, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.acl-long.353. URL <https://aclanthology.org/2023.acl-long.353>.
- Nawrot, P., Łańcucki, A., Chochowski, M., Tarjan, D., and Ponti, E. Dynamic memory compression: Retrofitting LLMs for accelerated inference. In *Forty-first International Conference on Machine Learning*, 2024. URL <https://openreview.net/forum?id=tDRYrAkOB7>.
- Pagnoni, A., Pasunuru, R., Rodriguez, P., Nguyen, J., Muller, B., Li, M., Zhou, C., Yu, L., Weston, J. E., Zettlemoyer, L., Ghosh, G., Lewis, M., Holtzman, A., and Iyer, S. Byte latent transformer: Patches scale better than tokens. In Che, W., Nabende, J., Shutova, E., and Pilehvar, M. T. (eds.), *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 9238–9258, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.453. URL <https://aclanthology.org/2025.acl-long.453/>.
- Peharz, R., Tschitschek, S., Pernkopf, F., and Domingos, P. M. On Theoretical Properties of Sum-Product Net-

- works. In *International Conference on Artificial Intelligence and Statistics*, 2015.
- Peharz, R., Lang, S., Vergari, A., Stelzner, K., Molina, A., Trapp, M., Van Den Broeck, G., Kersting, K., and Ghahramani, Z. Einsum networks: fast and scalable learning of tractable probabilistic circuits. In *Proceedings of the 37th International Conference on Machine Learning, ICML’20. JMLR*, 2020a.
- Peharz, R., Vergari, A., Stelzner, K., Molina, A., Shao, X., Trapp, M., Kersting, K., and Ghahramani, Z. Random sum-product networks: A simple and effective approach to probabilistic deep learning. In *35th Conference on Uncertainty in Artificial Intelligence (UAI)*, volume 115 of *Proceedings of Machine Learning Research*, pp. 334–344. PMLR, 2020b.
- Peharz, R., Vergari, A., Stelzner, K., Molina, A., Shao, X., Trapp, M., Kersting, K., and Ghahramani, Z. Random sum-product networks: A simple and effective approach to probabilistic deep learning. In Adams, R. P. and Gogate, V. (eds.), *Proceedings of The 35th Uncertainty in Artificial Intelligence Conference*, volume 115 of *Proceedings of Machine Learning Research*, pp. 334–344. PMLR, 22–25 Jul 2020c. URL <https://proceedings.mlr.press/v115/peharz20a.html>.
- Rumbelow, J. and Watkins, M. Solidgoldmagikarp (plus, prompt generation). <https://www.lesswrong.com/posts/aPeJE8bSo6rAFoLqg/solidgoldmagikarp-plus-prompt-generation> (accessed 11-May-2025), 2023.
- Shao, X., Molina, A., Vergari, A., Stelzner, K., Peharz, R., Liebig, T., and Kersting, K. Conditional sum-product networks: Imposing structure on deep probabilistic architectures. In *International Conference on Probabilistic Graphical Models*, pp. 401–412. PMLR, 2020.
- Shao, X., Molina, A., Vergari, A., Stelzner, K., Peharz, R., Liebig, T., and Kersting, K. Conditional sum-product networks: Modular probabilistic circuits via gate functions. *International Journal of Approximate Reasoning*, 140:298–313, 2022.
- Shpilka, A. and Yehudayoff, A. Arithmetic circuits: A survey of recent results and open questions. *Foundations and Trends in Theoretical Computer Science*, 5:207–388, 2010.
- Stern, M., Shazeer, N. M., and Uszkoreit, J. Blockwise parallel decoding for deep autoregressive models. In *Neural Information Processing Systems (NeurIPS)*, 2018.
- Sun, Z., Suresh, A. T., Ro, J. H., Beirami, A., Jain, H., and Yu, F. Spectr: Fast speculative decoding via optimal transport. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=SdYHLTCC5J>.
- The april Lab. cirkit, October 2024. URL <https://github.com/april-tools/cirkit>.
- Varin, C., Reid, N., and Firth, D. An overview of composite likelihood methods. *Statistica Sinica*, 21, 01 2011.
- Vergari, A., Di Mauro, N., and Esposito, F. Visualizing and understanding sum-product networks. *Machine Learning*, 108(4):551–573, 2019a.
- Vergari, A., Di Mauro, N., and Van den Broeck, G. Tractable probabilistic models: Representations, algorithms, learning, and applications. *Tutorial at the 35th Conference on Uncertainty in Artificial Intelligence (UAI)*, 2019b.
- Vergari, A., Choi, Y., Liu, A., Teso, S., and Van den Broeck, G. A compositional atlas of tractable circuit operations for probabilistic inference. In *Advances in Neural Information Processing Systems 34 (NeurIPS)*, pp. 13189–13201. Curran Associates, Inc., 2021.
- Wang, J., Gangavarapu, T., Yan, J. N., and Rush, A. M. Mambabyte: Token-free selective state space model. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=X1xNsukKssb>.
- Wertheimer, D., Rosenkranz, J., Parnell, T., Suneja, S., Ranganathan, P., Ganti, R., and Srivatsa, M. Accelerating production llms with combined token/embedding speculators, 2024. URL <https://arxiv.org/abs/2404.19124>.
- Xia, H., Yang, Z., Dong, Q., Wang, P., Li, Y., Ge, T., Liu, T., Li, W., and Sui, Z. Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding. In Ku, L.-W., Martins, A., and Srikumar, V. (eds.), *Findings of the Association for Computational Linguistics ACL 2024*, pp. 7655–7671, Bangkok, Thailand and virtual meeting, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.456. URL <https://aclanthology.org/2024.findings-acl.456>.
- Yang, Z., Dai, Z., Salakhutdinov, R., and Cohen, W. W. Breaking the softmax bottleneck: A high-rank RNN language model. In *International Conference on Learning Representations*, 2018. URL <https://openreview.net/forum?id=HkwZSG-CZ>.
- Yidou-Weng, G., Li, I., Liu, A., Broadrick, O., Cui, Y., den Broeck, G. V., and Wang, B. Learning tractable distributions of language model continuations, 2026. URL <https://arxiv.org/abs/2511.16054>.

- Yu, L., Simig, D., Flaherty, C., Aghajanyan, A., Zettlemoyer, L., and Lewis, M. Megabyte: Predicting million-byte sequences with multiscale transformers. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *Advances in Neural Information Processing Systems*, volume 36, pp. 78808–78823. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/f8f78f8043f35890181a824e53a57134-Paper-Conference.pdf.
- Zhang, H., Dang, M., Peng, N., and Van Den Broeck, G. Tractable control for autoregressive language generation. In Krause, A., Brunskill, E., Cho, K., Engelhardt, B., Sabato, S., and Scarlett, J. (eds.), *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 40932–40945. PMLR, 23–29 Jul 2023. URL <https://proceedings.mlr.press/v202/zhang23g.html>.
- Zhang, H., Kung, P.-N., Yoshida, M., Van den Broeck, G., and Peng, N. Adaptable logical control for large language models. In Globerson, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J., and Zhang, C. (eds.), *Advances in Neural Information Processing Systems*, volume 37, pp. 115563–115587. Curran Associates, Inc., 2024a. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/d15c16cf5619a2b1606da5fc88e3f1a9-Paper-Conference.pdf.
- Zhang, H., Dang, M., Wang, B., Ermon, S., Peng, N., and den Broeck, G. V. Scaling probabilistic circuits via monarch matrices. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=uzVShD7wHO>.
- Zhang, J., Wang, J., Li, H., Shou, L., Chen, K., Chen, G., and Mehrotra, S. Draft & verify: Lossless large language model acceleration via self-speculative decoding. In Ku, L.-W., Martins, A., and Srikumar, V. (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 11263–11282, Bangkok, Thailand, August 2024b. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.607. URL <https://aclanthology.org/2024.acl-long.607/>.
- Zheng, L., Zhao, X., Wang, G., Wu, C., Dong, D., Wang, A., Wang, M., Du, Y., Bo, H., Sharma, A., Li, B., Zhang, K., Hu, C., Thakker, U., and Kong, L. Evabyte: Efficient byte-level language models at scale, 2025. URL <https://hkunlp.github.io/blog/2025/evabyte>.

A. Notation

We adapt notation and nomenclature from the tensor factorisation (Kolda & Bader, 2009) and circuit (Loconte et al., 2025a) literature.

We denote ordered sets of random variables with $\mathbf{X}$, $\mathbf{Y}$ and $\mathbf{Z}$, and we use $[n]$ to express the set $\{1, 2, \dots, n\}$ with $n > 0$. The domain of a variable X is denoted as $\text{dom}(X)$, and we denote as $\text{dom}(\mathbf{X}) = \text{dom}(X_1) \times \dots \times \text{dom}(X_n)$ the joint domain of variables $\mathbf{X} = \{X_i\}_{i=1}^n$. We denote scalars with lower-case letters (e.g., $a \in \mathbb{R}$), vectors with boldface lower-case letters (e.g., $\mathbf{a} \in \mathbb{R}^N$), matrices with boldface upper-case letters (excluding those used for variables, e.g., $\mathbf{A} \in \mathbb{R}^{M \times N}$), and tensors with boldface calligraphic letters (e.g., $\mathcal{A} \in \mathbb{R}^{I_1 \times I_2 \times I_3}$). Moreover, we use subscripts to denote entries of tensors (e.g., a_{ijk} is the (i, j, k) -th entry in $\mathcal{A}$).

B. Background on circuits

Circuits have a long history in theoretical computer science (Shpilka & Yehudayoff, 2010) and probabilistic reasoning (Darwiche, 2003; 2009). In their more modern definition and application to machine learning (Vergari et al., 2019b; Choi et al., 2020), circuits are introduced as structured computational graphs, simplified neural networks where one is allowed to use units from a restricted set of neurons (sum, product and input units) and whose connections need to abide certain *structural properties* to guarantee tractability (Choi et al., 2020; Vergari et al., 2021), as discussed next.

B.1. Structural properties

Tractability is to be intended as the ability to exactly compute a given function (operation) over the circuit in time that is polynomial in its size, denoted as $|c|$ for a circuit c , and representing the number of edges between the computational units. For example, a circuit c can exactly integrate *any subset of variables* in time $\mathcal{O}(|c|)$ if (i) its input functions can be integrated efficiently and (ii) it is *smooth* and *decomposable* (Darwiche & Marquis, 2002; Choi et al., 2020).

Definition 1 (Smoothness and decomposability (Darwiche & Marquis, 2002; Choi et al., 2020)). A circuit is *smooth* if for every sum unit n , all its input units depend on the same variables, i.e., $\forall i, j \in \text{in}(n) : \text{sc}(i) = \text{sc}(j)$. A circuit is *decomposable* if the distinct inputs of every product unit n depend on disjoint sets of variables, i.e., $\forall i, j \in \text{in}(n) \ i \neq j : \text{sc}(i) \cap \text{sc}(j) = \emptyset$.

Note that all the PC architectures we have discussed in this paper, **FF**, **CP**, **HMM** and **BTree**, are smooth and decomposable circuits by construction (Peharz et al., 2020c; Loconte et al., 2025a). The reader is encouraged to check this by themselves for the architectures in Fig. 2. Exactly integrating variables out is relevant to compute marginals such as the normalisation constant of the distribution encoded by the circuit (Eq. (3)). Note that in our implementation, circuits are normalised by design (Peharz et al., 2015), as we assume that input distributions are normalised categoricals and all sum units form a convex combination as their weights are parameterised with a softmax function (see Sec. 3.3).

More importantly for our MTPCs, we can draw samples efficiently from the distribution of a circuit that is both smooth and decomposable, as we discuss in the next sub-section.

B.2. Sampling a circuit

A smooth and decomposable PC can use ancestral sampling to generate a complete sample for all n tokens in a window. In a nutshell, we can iteratively sample each latent variable in the hierarchy encoded by the PC, and then sample the selected input distributions, in the same way one sample one (hierarchical) mixture model by first sampling one component and then drawing a sample from that component.

Operationally, Alg. 1 details the procedure. We have to sample one input branch for each sum unit we encounter when performing a backward traversal of the circuit computational graph (from the circuit output back to the input distributions). Such a branch is sampled proportionally to the sum unit weights ω_j , which encode the mixture components (or equivalently the transition probabilities in an HMM). Then, when we traverse a product unit, we follow all its input branches. When we reach an input unit, we sample a token proportionally to the parameters ϕ_{ij} of the categorical distributions encoded in the unit. If the circuit is smooth and decomposable, by this process we are guaranteed to end up in a set of input units whose scope is the full set of tokens $\mathbf{X}$ and in which only one input unit is selected per token position i (line 13 of Alg. 1). This procedure can be tensorised to efficiently generate a batch of samples in a single pass over the computational graph of the circuit (Vergari et al., 2019a; Peharz et al., 2020b;a; Loconte et al., 2025a; Liu et al., 2024).

Algorithm 1 SAMPLE(c)

Input: A smooth, decomposable and normalised PC c encoding a joint distribution q over the next n tokens $\mathbf{X} = \{X_1, \dots, X_n\}$ **Output:** a sample $\mathbf{x} \sim q(\mathbf{X})$.

```

1:  $\mathbf{x} \leftarrow \text{zeroes}(n)$  {init empty sample}
2:  $c_n \leftarrow \text{output}(c)$ 
3:  $\mathcal{N} \leftarrow \text{queue}(\{c_n\})$  {traverse the computational graph from outputs to inputs}
4: while  $\mathcal{N}$  not empty do
5:    $c_n \leftarrow \text{pop}(\mathcal{N})$  { $c_n$  is a sum unit}
6:   if  $c_n = \sum_{j=1}^r \omega_j c_j$  then
7:      $k \leftarrow \text{sampleCategorical}(\omega_1, \dots, \omega_r)$  {sample from a categorical with  $r$  states}
8:      $\mathcal{N} \leftarrow \text{push}(\mathcal{N}, c_k)$ 
9:   else if  $c_n = \prod_{j=1}^d c_j$  then
10:    for  $k = 1 \dots d$  do
11:       $\mathcal{N} \leftarrow \text{push}(\mathcal{N}, c_k)$  {visit all inputs of product unit  $c_n$ }
12:    end for
13:   else if  $c_n$  is an input unit over variable  $X_i$  and parameters  $\phi_i$  then
14:      $x_i \leftarrow \text{sampleCategorical}(\phi_i)$  {sample from a categorical with  $v = |\mathcal{V}|$  states}
15:   end if
16: end while
17: return  $\mathbf{x}$ 

```

Lastly, we remark that this routine is potentially computationally more efficient than the one implemented in [Basharin et al. \(2025\)](#), as the latter is based on autoregressive inverse transform sampling (see [Loconte et al. \(2024\)](#) for a discussion) and requires sampling one token at a time.

C. Complexity Analysis for Circuits in MTPC

We computed the theoretical number of FLOPs of the multi-token prediction linear head and the circuit feed-forward evaluation, by using [calculate-flops.pytorch](#). We show the FLOPs needed to evaluate the circuit since it is required to compute the likelihoods in the speculative decoding algorithm. In Table 2, we report the number of parameters for combinations of circuits and the maximum number of draft tokens. We set the number of components of each sum to $r = 32$ in CP, HMM, and BTree. We also show the FLOPs required to do prefilling of a very small sequence of bytes (of length 64) with EvaByte for reference.

While using the MTPC-BTREE or MTPC-HMM increases the FLOPs to evaluate the circuit when compared to the fully factorised circuit, it is still orders of magnitudes smaller than the cost of evaluating the LLM trunk (i.e., a few thousands vs billions). Moreover, using the BTree or HMM substantially increases the parameters of the multi-token head linear projection, when compared to the fully factorised circuit. However, we note that it is still very close to the number of parameters required by MTPC-CP (i.e., 730 M vs 671 M in the case of 16 draft tokens). This is because the categorical input units account for the same vast majority of circuit parameters in all circuits except for the fully factorised one.

Table 2. FLOPs and parameter counts for circuit and MTP head configurations.

LLM	FLOPs	# Tokens	Circuit	FLOPs	Parameters	MTP Head	FLOPs	Parameters
EvaByte	828.95 G	8	FF	7	2,560	Linear	0.02 G	10 M
			CP	65	81,952	Linear	0.67 G	336 M
			HMM	14.4 K	89,120	Linear	0.73 G	365 M
			BTree	12.4 K	88,096	Linear	0.72 G	361 M
		16	FF	15	5,120	Linear	0.04 G	21 M
			CP	65	163,872	Linear	1.34 G	671 M
			HMM	30.8 K	179,232	Linear	1.47 G	734 M
			BTree	28.7 K	178,208	Linear	1.46 G	730 M

D. Shallow Autoregressive Models for Byte-Level LLMs

We omit shallow autoregressive models such as Hydra (Ankner et al., 2024) and EAGLE (Li et al., 2024) from our setup as they do not fit our desiderata we mentioned in the introduction. At the same time, one may ask how well shallow autoregressive models perform for byte-level LLMs. While shallow autoregressive models are state-of-the-art for subword-level models, their superiority does not obviously transfer to byte-level models, and fairly comparing models from MTPC with them is non-trivial, as we discuss below.

D.1. Key Differences between Byte-Level LLMs and Subword-Level LLMs

Byte-level models differ from subword-level models in two important ways: a) they need a larger MTP window size and b) they have a much smaller vocabulary size. These two differences alter the balance in parameter allocation and make it non-obvious that Hydra and EAGLE are state-of-the-art methods for byte-level LLMs (this remains to be seen).

Window size differences. To match subword-level throughput, byte-level models require MTP window sizes n roughly 4-5 times longer, since 1 BPE token $\approx$ 4-5 bytes for English. Importantly, the memory required for the d -dimensional feature vectors for Hydra/Eagle scales as $\mathcal{O}(n \times s \times d)$, as the feature vector is adapted per token in the MTP window, in contrast to our MTPC feature embeddings which scale as $\mathcal{O}(s \times d)$. As such, when n is larger, shallow autoregressive models are less appealing for MTP.

Vocabulary differences. Moreover, byte-level vocabularies are 100 - 400 $\times$ smaller (e.g. $V_b=320$ vs V_s in 32k - 128k), while d is comparable (e.g. $d = 4096$ for EvaByte and $d = 3072$ for Llama). Using $d = 3200$ for simplicity, the logit-to-feature ratio for subword-level models is $V_s/D > 10$, while for byte models we have $(V_b/D < 1/10)$. Thus, increasing expressivity in logit-space (as MTPC does) can be more efficient for byte-level models than increasing it in feature-space as done in Hydra/EAGLE.

D.2. Comparing MTPC to Shallow Autoregressive Models is non-trivial

While we cannot make strong claims without experiments, our paper shows the expressiveness/latency trade-off is sensitive even to small changes, let alone differences of the magnitude mentioned above for the vocabulary size. This suggests that significant optimisation of the EAGLE/Hydra architectures may be needed for a fair comparison to MTPC on byte-level LLMs. We believe that optimising shallow autoregressive models for byte-level models is interesting, but we leave this for future work.

E. Target Model Details

Table 3. Details of target models we retrofitted using MTPC.

	EvaByte	Llama3.2 3B Byte
MTP Window	8	—
# Transformer Layers	32	28
# Attention Heads	32	24
Embedding Size	4096	3072
dtype	bfloat16	float32
Vocab Size	320	268
Number of Parameters	6.5B	3B
Max Context Window	32768	131072

E.1. EvaByte Details

In Table 4 we provide a copy of the benchmark results of fine-tuned version of EvaByte, EvaByte-SFT, which we retrofit in the paper. The model card can be found at <https://hf.co/EvaByte/EvaByte-SFT>, see also Zheng et al. (2025) for more details. The numbers in the table above were produced by Medusa-style (Cai et al., 2024) tree-based typical decoding. For all benchmarks apart from HumanEval, the results were produced via greedy decoding, *i.e.*, similar to (Stern

Table 4. Downstream benchmark performance of EvaByte-SFT, table taken verbatim from Zheng et al. (2025). Entries with † were computed by the EvaByte authors. All numbers were computed with Medusa-style tree-based greedy decoding using the multi-token head, apart from HumanEval*, which used typical sampling. The authors of EvaByte followed Tulu 3, and evaluated the Pass@10 rate for HumanEval with 20 samples at temperature 0.8.

Model	BBH	GSM8k	IFEval	MATH	MMLU	HumanEval*	TruthQA
Gemma-2-9B-it	20.0	79.7	69.9	29.8	69.1	71.7	61.4
Ministral-8B-Instruct	56.2	80.0	56.4	40.0	68.5	91.0	55.5
Qwen-2.5-7B-Instruct	25.3	83.8	74.7	69.9	76.6	93.1	63.1
Llama-3.1-8B-Instruct	69.7	83.4	80.6	42.5	71.3	86.3	55.1
Tulu 3 8B	66.0	87.6	82.4	43.7	68.2	83.9	55.0
OLMo-7B-Instruct	35.3	14.3	32.2	2.1	46.3	28.7†	44.5
OLMo-v1.7-7B-Instruct	34.4	23.2	39.2	5.2	48.9	49.7†	55.2
OLMoE-1B-7B-0924-Instruct	37.2	47.2	46.2	8.4	51.6	54.8	49.1
MAP-Neo-7B-Instruct	26.4	69.4	35.9	31.5	56.5	72.1†	51.6
OLMo-2-7B-SFT	50.7	71.2	68.0	25.1	62.0	67.0†	47.8
OLMo-2-7B-1124-Instruct	48.5	85.2	75.6	31.3	63.9	67.6†	56.3
EvaByte-SFT	34.6	52.9	60.2	29.8	49.5	73.7	46.3

et al., 2018) with the exception of producing the last “free” token. As such, the produced tokens are equivalent to what EvaByte-STP would produce, and the authors found that the metrics were the same with EvaByte-STP up to some small rounding errors. For HumanEval the authors used tree-based typical decoding, which in this case does not maintain the quality of the EvaByte-STP model. The details above were shared with us by Lin Zheng, the author of EvaByte.

F. Speculative Decoding

We give pseudocode for our self-speculative decoding algorithm below. The algorithm accepts between 0 and n tokens, but always generates between 1 and n tokens, where n is the MTP window size. The algorithm is very similar to vanilla speculative decoding (Leviathan et al., 2022), but our algorithm includes a modification that reduces latency for the self-speculative scenario, and for this it needs to sacrifice the last “free” token typically obtained from the verifier. The gain in latency is possible because we can evaluate the shared LLM once per draft/verification cycle, while a naive implementation of Leviathan et al. (2022) for self-speculative decoding would need two, approximately halving the possible throughput.

In our self-speculative setup, the verifier and draft LLMs share some layers of the backbone. Importantly, the verifier is always computing LLM states ahead of the draft. As such, we can get away with a single forward pass through the shared LLM, similar to Medusa (Cai et al., 2024), by re-using the LLM backbone state computed by the verifier for the draft model. For this to work, we cannot accept a “last sample for free” from the verifier (lines 23-30) Alg. 3), as we would not have the backbone state for this new token and it is not worth paying an extra LLM evaluation for it. Therefore, in our algorithm we only sample the “free” token from the verifier in the rare case that no tokens are accepted. This is necessary because the model can get caught in successive no-accept states in the sampling case, or get stuck in an infinite loop if we use greedy decoding. If any tokens were accepted, we use the last state of the shared backbone computed during the verify phase to seed the draft phase. In what follows, if we have no LoRA layers, the algorithm is modified to have a single component: the shared encoder.

Algorithm 2 SHAREDSTATESELFSPECULATIVEDECODING

Architecture Components:

Three components: Shared Encoder (S), Verifier (V), Draft (D)
Each with their own KV-cache

Given: A prompt of length L

Initialisation:

Prefill V to $L - 1$, and D and S to L
Set S and D state

Switch to draft/verify cycle:

while true **do**

Draft stage:

if S state is not set **then**

 Compute S by conditioning on the additional token

end if

Use S state to compute D state

Parameterize MTPC with D state

Draft n tokens

Verify stage:

Compute S state on $n + 1$ tokens (draft + predecessor)

Compute V state using S state

Obtain up to $n + 1$ tokens from speculative decoding

if 0 tokens accepted **then**

 Keep “free” token sampled from last valid logits

 Unset S state (stale)

else

 Accept n tokens (drop “free” token)

 Set S state (hidden state for last accepted token)

end if

end while

Algorithm 3 SELFSPECULATIVEDECODING($\mathbf{x}_{\leq t}, f, h, c, g$)

Input: A prefix $\mathbf{x}_{\leq t}$ of length t , an LLM backbone $f: \mathcal{V}^* \rightarrow \mathbb{R}^d$, an LLM head $h: \mathbb{R}^d \rightarrow \Theta$ parameterising a PC c encoding a joint distribution q over the next n tokens, and an LLM head $g: \mathbb{R}^d \rightarrow \Delta^v$ computing the next token probabilities.

Output: A sentence $(\mathbf{x}_{\leq t} \parallel \mathbf{x}_{t+1:t+s}) \in \mathcal{V}^{t+s}$ where $1 \leq s \leq n+1$. Moreover, we have that $\mathbf{x}_{t+1:t+s} \sim p(x_{t+1}, \dots, x_{t+s} \mid \mathbf{x}_{\leq t})$ as equivalently encoded by the autoregressive single token prediction model consisting of f and g only (Leviathan et al., 2022).

```

1:  $\mathbf{e}_t \leftarrow f(\mathbf{x}_{\leq t})$  {Compute the last embedding}
2:  $\theta \leftarrow h(\mathbf{e}_t)$  {Compute the circuit parameters}
3:
4: Let  $q(\mathbf{X}_{t+1:t+n} \mid \mathbf{x}_{\leq t}) = \frac{1}{Z_\theta} c(\mathbf{X}_{t+1:t+n} \mid \theta)$ 
5:  $\mathbf{x}_{t+1:t+n} \sim q(\mathbf{X}_{t+1:t+n} \mid \mathbf{x}_{\leq t})$  {Sample  $n$  tokens from  $c$  in time  $\mathcal{O}(|c|)$ }
6:  $\mathbf{x} \leftarrow \mathbf{x}_{\leq t} \parallel \mathbf{x}_{t+1:t+n}$  {Concatenate the prefix with the  $n$  tokens}
7:
8: Compute in parallel for  $1 \leq i \leq n$ : {Compute marginals in time  $\mathcal{O}(|c|)$ }
9:    $q(\mathbf{x}_{t+1:t+i} \mid \mathbf{x}_{\leq t}) = \sum_{x_{t+i+1}, \dots, x_{t+n} \in \mathcal{V}} q(\mathbf{x}_{t+1:t+n} \mid \mathbf{x}_{\leq t})$ 
10:
11: Compute in parallel for  $1 \leq i \leq n+1$ : {Compute target model conditionals}
12:    $p(X_{t+i} \mid \mathbf{x}_{\leq t+i-1}) = g(\mathbf{e}_{t+i-1})$ , where  $\mathbf{e}_{t+i-1} = f(\mathbf{x}_{\leq t+i-1})$ 
13:
14:  $s \leftarrow 0$  {Determine the number of accepted tokens  $s$ ,  $0 \leq s \leq n$ }
15: while  $s < n$  do
16:    $\alpha \sim \mathcal{U}(0, 1)$ 
17:   if  $s > 0$  then
18:      $q(x_{t+s+1} \mid \mathbf{x}_{\leq t+s}) \leftarrow q(\mathbf{x}_{t+1:t+s+1} \mid \mathbf{x}_{\leq t}) / q(\mathbf{x}_{t+1:t+s} \mid \mathbf{x}_{\leq t})$ 
19:   end if
20:   if  $\alpha > p(x_{t+s+1} \mid \mathbf{x}_{t+s}) / q(x_{t+s+1} \mid \mathbf{x}_{t+s})$  then
21:     exit loop
22:   end if
23:    $s \leftarrow s + 1$ 
24: end while {Sample one last token from the autoregressive LLM model}
25: if  $s < n$  then
26:   Let  $s(X_{t+s+1}) = q(X_{t+s+1} \mid \mathbf{x}_{\leq t+s})$  {Adjust the distribution first, if we accept fewer tokens}
27:   Let  $m(X_{t+s+1}) = \max(0, p(X_{t+s+1} \mid \mathbf{x}_{\leq t+s}) - s(X_{t+s+1}))$ 
28:    $r(X_{t+s+1} \mid \mathbf{x}_{t+s}) = m(X_{t+s+1}) / Z$ , with  $Z = \sum_{x' \in \mathcal{V}} m(x')$ 
29:    $x_{t+s+1} \sim r(X_{t+s+1} \mid \mathbf{x}_{\leq t+s})$ 
30: else
31:    $x_{t+s+1} \sim p(X_{t+s+1} \mid \mathbf{x}_{\leq t+s})$ 
32: end if
33: return  $\mathbf{x}_{\leq t+s} \parallel x_{t+s+1}$ 

```

F.1. GPU Memory Usage For Speculative Decoding

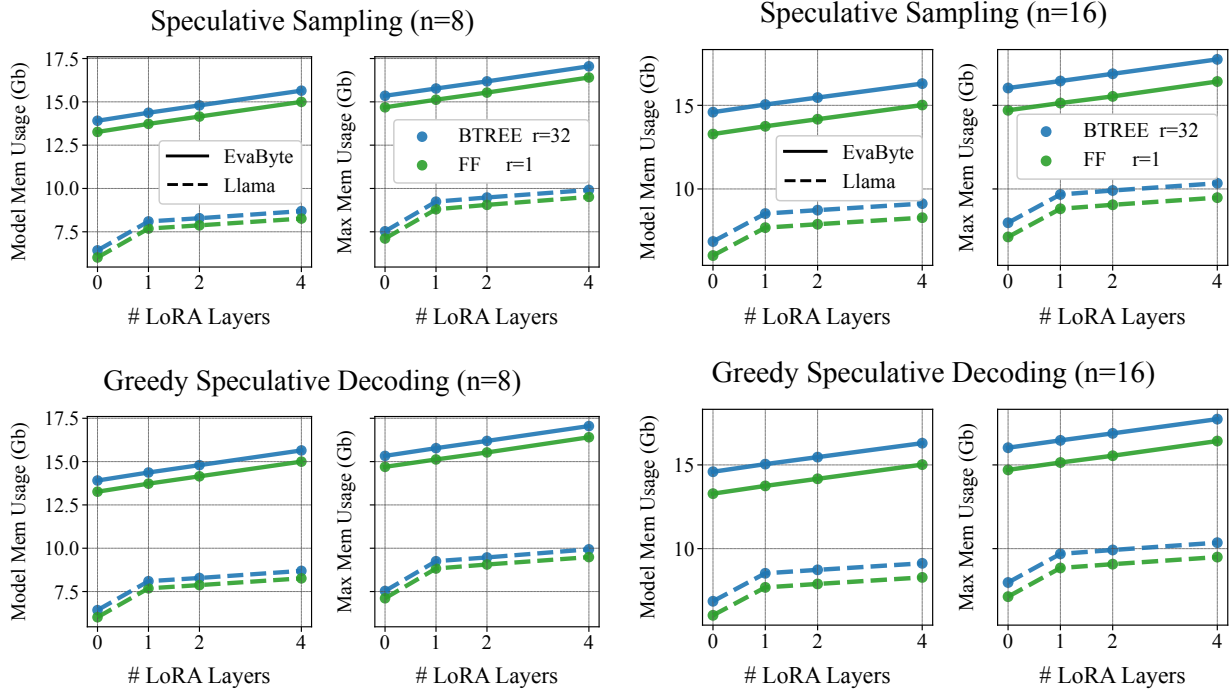


Figure 5. GPU memory usage as a function of the number of LoRA layers (x-axis) during model loading (left subplot) and maximum memory usage during inference (right subplot). As can be seen, MTPC-BTREE has similar memory requirements to MTPC-FF and scales gracefully with the MTP window size, n . Moreover, retrofitting LoRA on the last 1-4 layers barely changes the footprint: we pay a memory overhead of approximately k transformer layers when using k LoRA layers. Measurements taken on the L40S GPUs during the speculative decoding experiments with KV cache enabled, BF16 computations, and 1,024 generated tokens.

G. Memory Bottleneck Due to Logits

In this section we elaborate on the memory bottleneck that arises in MTP, especially when we use a model that has mixture coefficients, *e.g.* MTPC-CP, and propose ways to alleviate this bottleneck. As we mentioned in Sec. 5, the GPU memory required during training to store the logits scales linearly in the vocabulary size as $s \times n \times v \times r$, where s is the context length, n the MTP window length, v the vocabulary size and r the number of mixture components. As can be seen in Table 5, the size of the logits quickly becomes too large to fit in GPU memory if we scale the vocabulary size. Therefore, although we can scale MTPC to subword-level models with vocabulary sizes of 32k+, we are forced to use a much smaller number of mixture coefficients, making the approach less attractive. However, there are several strategies to keep the parameter count under control despite the increase in vocabulary size from 320 to 32k+ which can be explored in follow-up work.

Subsampling MTP windows. One pragmatic way of reducing the memory requirements due to logits during training is to avoid realising all of them. For example, we could subsample the token windows used for training in the MTP window.

Unembedding parameter sharing. We could share parameters across token positions and the mixture components and fine-tune low-rank adaptors (Hu et al., 2022) to allow some flexibility while keeping the parameter count under control. For example, we could parametrise the unembedding matrix for token i and mixture component j as $\mathbf{U}_{i,j} = \mathbf{W}_i + \mathbf{L}_{i,j}$, where $\mathbf{W}_i \in \mathbb{R}^{v \times d}$ is the pretrained unembedding matrix for token i and $\mathbf{L}_{i,j} = \mathbf{A}_{i,j} \mathbf{B}_{i,j}$ is a rank- k LoRA adaptor where $\mathbf{A}_{i,j} \in \mathbb{R}^{v \times k}$, $\mathbf{B}_{i,j} \in \mathbb{R}^{k \times d}$. For EvaByte, the pretrained unembedding matrices are available and we smoothly resume training from them by setting all entries of $\mathbf{B}_{i,j}$ to zero, while for other models we would need to randomly initialise the unembedding matrices.

Low-rank unembedding matrices. Another option is to explore whether we can lower the rank of the unembedding matrices (*i.e.* make d smaller) without losing performance. While it is known that a low-rank softmax layer is less

Table 5. Memory footprint of the logits for MTPC-CP as a function of the vocabulary size v , the MTP window size n , and the number of mixture coefficients r for a sequence length of 8192. While the logits are manageable for a byte-level model with a vocabulary of size $v = 320$ even for large r , memory scales linearly in v , so for subword-level models with large vocabularies the requirement quickly exceeds that available on our GPU (80 GB).

v	r	Size (float 32 precision)	
		$n = 8$	$n = 16$
320	1	80.0 MB	160.0 MB
	4	320.0 MB	640.0 MB
	8	640.0 MB	1.2 GB
	16	1.2 GB	2.5 GB
	32	2.5 GB	5.0 GB
32,000	1	7.8 GB	15.6 GB
	4	31.2 GB	62.5 GB
	8	62.5 GB	125.0 GB
	16	125.0 GB	250.0 GB
	32	250.0 GB	500.0 GB
128,000	1	31.2 GB	62.5 GB
	4	125.0 GB	250.0 GB
	8	250.0 GB	500.0 GB
	16	500.0 GB	1000.0 GB
	32	1.0 TB	2.0 TB

expressive (Yang et al., 2018), the solution proposed in Yang et al. (2018, Section 2.4) is to increase expressivity by using a mixture of softmaxes. Since we are already employing a mixture of softmaxes (albeit across multiple tokens), we believe it may be possible to shrink d without sacrificing too much performance.

Hierarchical partitioning and tensor factorisations. Lastly, we can explore ways of partitioning large vocabularies, *e.g.* hierarchical softmax (Mnih & Hinton, 2008), which introduces a non-linearity or other multi-linear tensor factorisations. For example, we could partition the vocabulary for each token into m categories and introduce a latent variable to switch between the categories for each token position. This can be beneficial because within each category we can limit the active vocabulary size significantly, and can therefore use an unembedding matrix which is lower rank, as proposed previously.

H. Hidden Markov Models Setup

In our experiments we use **contextual, inhomogeneous** hidden Markov models (HMMs) with **identity initialisation** (see Table 6). We chose the above after preliminary experiments where we assessed the following configuration choices for training our HMMs.

Parameterisation. We can parameterise HMMs to either be contextual, *i.e.* we can make the transition probabilities depend on the input, or we can make the transition probabilities be independent of the input (non-contextual).

Transition type. The transition matrix can be the same at each time step (homogeneous) or it can be different (inhomogeneous). The former would correspond to additional parameter sharing across sum layers in the circuit representation. We note that inhomogeneous HMMs subsume homogeneous HMMs. This is because inhomogeneous HMMs could in theory learn parameters that do not vary from timestep to timestep, thus becoming equivalent to a homogeneous HMM.

Initialisation. A crucial setup is to initialise the HMM with transition matrices that are identity matrices, which make the HMM equivalent to CP at the beginning of training. We achieve this by adding a bias term to allow the HMM model to be initialised to identity matrices. This setting in combination with extending to larger token windows, *i.e.* $n = 16$ lead to a scenario where HMMs outperform CP. The other alternative is to initialise the transition matrices uniformly at random (before softmax), but this complicates learning and yields performance that is lower than CP models.

Table 6. Most Successful HMM Configuration

Parameterisation		Transition Type		Initialisation	
Contextual	Non-Contextual	Homogeneous	Inhomogeneous	Identity Init.	Uniform Init.
✓	✗	✗	✓	✓	✗

I. Further Experimental Details

To make the comparison between methods fair, we: a) constrained the models to not produce end-of-sequence symbols during generation, as the latency of retrieving KV cache items from memory increases with sequence length (Nawrot et al., 2024) and b) we filtered the validation set of the models to only include examples with both prompts and responses in English, as acceptance rates may vary dramatically based on the language chosen for the response.

We compute throughput by generating answers to 250 prompts and report the mean and std of 3 runs with different prompts.

J. Alternative Losses

In early versions of this work we also experimented using a Kullback-Leibler divergence (KL) loss as recommended by Basharin et al. (2025). However, we found that training with the KL loss doubled the training time while requiring a lot more memory, and the benefits in the number of accepted tokens did not outweigh the additional complexity. For completeness we include the loss below. **KL Loss** $\mathcal{L}$

$$\mathcal{L} = \sum_{j=1}^n \mathcal{L}_j \gamma^{j-1}, \quad \mathcal{L}_j = \sum_{i=1}^N \sum_{t=1}^L \frac{f_{\text{KL}} \left(p_{\theta}(x_{t+j}^{(i)} \mid \mathbf{x}_{<t+j}^{(i)}) \parallel q_{\theta'}(x_{t+j}^{(i)} \mid \mathbf{x}_{<t+j}^{(i)}) \right)}{N_{\text{valid}}(i, j)} \quad (6)$$

In the above we condition both the draft model, $q_{\theta'}$, and the target model, p_{θ} , on the gold data. The above is equivalent to the KL term from the word-level distillation loss in (Kim & Rush, 2016).¹¹

¹¹While performing sequence-level distillation, *i.e.*, conditioning on data sampled from the teacher model may improve distillation (Kim & Rush, 2016), we did not explore this.

K. Results on L40S GPU

K.1. EvaByte 6.5B

K.1.1. SPECULATIVE SAMPLING

Table 7

		$\odot \mu_{\text{lat}} \downarrow$	$\checkmark \mu_{\text{acc}} \uparrow$	$\mu_{\text{tok/s}} \uparrow$	$\max_{\text{tok/s}}$
FF	1	0.0299 ± 0.0003	5.19 ± 0.05	176.3 ± 0.9	297.55
	8	0.0314 ± 0.0005	5.67 ± 0.05	184.2 ± 3.9	297.92
	16	0.0323 ± 0.0020	5.79 ± 0.08	183.4 ± 13.3	290.71
CP	32	0.0314 ± 0.0003	5.88 ± 0.02	191.0 ± 1.6	287.88
	64	0.0327 ± 0.0015	5.96 ± 0.04	185.9 ± 7.7	281.10
	128	0.0337 ± 0.0001	6.02 ± 0.03	182.3 ± 1.1	264.80

Table 8

			$\odot \mu_{\text{lat}} \downarrow$	$\checkmark \mu_{\text{acc}} \uparrow$	$\mu_{\text{tok/s}} \uparrow$	$\text{⌘} \times_{\text{STP}}$
1	1	STP	0.0254	—	39.5	1.00
8	1	FF	0.0299 ± 0.0003	5.19 ± 0.05	176.3 ± 0.9	4.47
		HMM	0.0362 ± 0.0006	6.02 ± 0.02	169.4 ± 2.6	4.29
	32	BTree	0.0326 ± 0.0008	6.02 ± 0.05	188.7 ± 6.1	4.78
		CP	0.0314 ± 0.0003	5.88 ± 0.02	191.0 ± 1.6	4.84
16	1	FF	0.0308 ± 0.0003	5.34 ± 0.03	176.1 ± 2.3	4.46
		HMM	0.0421 ± 0.0022	6.93 ± 0.06	168.0 ± 9.4	4.26
	32	CP	0.0333 ± 0.0009	6.18 ± 0.02	189.4 ± 5.4	4.80
		BTree	0.0347 ± 0.0000	6.76 ± 0.07	199.2 ± 2.2	5.05

Table 9

			$\odot \mu_{\text{lat}} \downarrow$	$\checkmark \mu_{\text{acc}} \uparrow$	$\mu_{\text{tok/s}} \uparrow$	$\text{⌘} \times_{\text{STP}}$
1	STP	—	0.0254	—	39.5	1.00
8	FF	0	0.0308 ± 0.0004	5.11 ± 0.03	168.9 ± 2.9	4.28
		1	0.0325 ± 0.0011	5.11 ± 0.07	160.0 ± 7.8	4.06
		2	0.0332 ± 0.0009	5.12 ± 0.07	157.1 ± 6.3	3.98
		4	0.0348 ± 0.0004	5.12 ± 0.04	149.5 ± 3.1	3.79
	BTree	0	0.0332 ± 0.0006	6.05 ± 0.05	186.1 ± 1.8	4.72
		1	0.0348 ± 0.0010	6.16 ± 0.06	180.9 ± 6.9	4.58
		2	0.0354 ± 0.0014	6.18 ± 0.03	178.2 ± 7.9	4.52
		4	0.0373 ± 0.0017	6.22 ± 0.01	170.9 ± 7.5	4.33
16	FF	0	0.0313 ± 0.0012	5.36 ± 0.07	173.8 ± 7.9	4.40
		1	0.0331 ± 0.0010	5.62 ± 0.08	172.6 ± 7.1	4.37
		2	0.0339 ± 0.0010	5.66 ± 0.15	170.5 ± 8.9	4.32
		4	0.0357 ± 0.0012	5.64 ± 0.11	161.2 ± 8.7	4.09
	BTree	0	0.0350 ± 0.0002	6.88 ± 0.10	200.7 ± 2.6	5.09
		1	0.0371 ± 0.0008	7.36 ± 0.03	203.1 ± 5.0	5.15
		2	0.0379 ± 0.0013	7.50 ± 0.03	202.5 ± 7.0	5.13
		4	0.0393 ± 0.0006	7.57 ± 0.13	197.1 ± 6.3	5.00

K.1.2. GREEDY SPECULATIVE DECODING

Table 10

		$\odot\mu_{\text{lat}} \downarrow$	$\boxtimes\mu_{\text{acc}} \uparrow$	$\mu_{\text{tok/s}} \uparrow$	$\text{max}_{\text{tok/s}}$
FF	1	0.0300 ± 0.0006	6.89 ± 0.01	235.4 ± 5.4	299.64
	8	0.0310 ± 0.0002	6.68 ± 0.00	221.5 ± 1.3	295.79
	16	0.0312 ± 0.0000	6.70 ± 0.02	221.3 ± 0.5	294.16
CP	32	0.0317 ± 0.0002	6.70 ± 0.01	217.4 ± 1.6	288.68
	64	0.0319 ± 0.0002	6.69 ± 0.01	215.9 ± 1.1	282.55
	128	0.0340 ± 0.0003	6.68 ± 0.02	202.3 ± 2.1	264.70

Table 11

			$\odot\mu_{\text{lat}} \downarrow$	$\boxtimes\mu_{\text{acc}} \uparrow$	$\mu_{\text{tok/s}} \uparrow$	$\boxtimes \times \text{STP}$
1	1	STP	0.0251	—	39.9	1.00
8	1	FF	0.0300 ± 0.0006	6.89 ± 0.01	235.4 ± 5.4	5.90
		HMM	0.0324 ± 0.0001	6.70 ± 0.03	212.8 ± 1.5	5.33
	32	CP	0.0317 ± 0.0002	6.70 ± 0.01	217.4 ± 1.6	5.45
		BTree	0.0315 ± 0.0004	6.71 ± 0.03	219.5 ± 3.6	5.50
16	1	FF	0.0299 ± 0.0002	7.75 ± 0.02	265.4 ± 1.8	6.65
		HMM	0.0360 ± 0.0009	8.43 ± 0.02	241.8 ± 6.0	6.06
	32	CP	0.0333 ± 0.0002	7.79 ± 0.05	242.2 ± 1.8	6.07
		BTree	0.0353 ± 0.0020	8.26 ± 0.08	242.6 ± 12.4	6.08

Table 12

			$\odot\mu_{\text{lat}} \downarrow$	$\boxtimes\mu_{\text{acc}} \uparrow$	$\mu_{\text{tok/s}} \uparrow$	$\boxtimes \times \text{STP}$
1	STP	—	0.0251	—	39.9	1.00
8	FF	0	0.0297 ± 0.0003	6.89 ± 0.01	237.3 ± 2.2	5.95
		1	0.0311 ± 0.0003	6.89 ± 0.01	227.2 ± 2.7	5.69
		2	0.0319 ± 0.0002	6.90 ± 0.02	221.2 ± 1.7	5.54
		4	0.0336 ± 0.0006	6.88 ± 0.01	209.6 ± 4.1	5.25
	BTree	0	0.0316 ± 0.0004	6.72 ± 0.02	218.6 ± 2.6	5.48
		1	0.0333 ± 0.0006	6.73 ± 0.01	208.3 ± 3.6	5.22
		2	0.0337 ± 0.0002	6.73 ± 0.01	205.6 ± 1.2	5.15
		4	0.0356 ± 0.0009	6.72 ± 0.03	194.5 ± 4.0	4.87
16	FF	0	0.0305 ± 0.0005	7.74 ± 0.03	260.6 ± 4.5	6.53
		1	0.0319 ± 0.0004	8.36 ± 0.04	269.2 ± 3.4	6.74
		2	0.0329 ± 0.0002	8.60 ± 0.02	269.1 ± 1.9	6.74
		4	0.0346 ± 0.0002	8.72 ± 0.02	259.9 ± 1.1	6.51
	BTree	0	0.0341 ± 0.0003	8.33 ± 0.10	253.2 ± 5.0	6.35
		1	0.0354 ± 0.0001	9.05 ± 0.06	265.2 ± 2.3	6.65
		2	0.0365 ± 0.0002	9.20 ± 0.09	261.2 ± 3.6	6.54
		4	0.0381 ± 0.0001	9.23 ± 0.07	251.3 ± 2.5	6.30

K.2. Llama 3.2 3B

K.2.1. SPECULATIVE SAMPLING

Table 13

		$\odot\mu_{\text{lat}} \downarrow$	$\boxtimes\mu_{\text{acc}} \uparrow$	$\mu_{\text{tok/s}} \uparrow$	$\max_{\text{tok/s}}$
FF	1	0.0291 $\pm$ 0.0031	1.71 $\pm$ 0.01	64.2 $\pm$ 6.4	342.39
	8	0.0298 $\pm$ 0.0031	1.92 $\pm$ 0.04	70.2 $\pm$ 5.9	355.81
CP	16	0.0301 $\pm$ 0.0036	1.98 $\pm$ 0.02	71.7 $\pm$ 8.0	357.04
	32	0.0311 $\pm$ 0.0032	2.04 $\pm$ 0.04	71.4 $\pm$ 8.2	357.82

Table 14

			$\odot\mu_{\text{lat}} \downarrow$	$\boxtimes\mu_{\text{acc}} \uparrow$	$\mu_{\text{tok/s}} \uparrow$	$\boxtimes \times_{\text{STP}}$
1	1	STP	0.0256	—	39.2	1.00
8	1	FF	0.0291 $\pm$ 0.0031	1.71 $\pm$ 0.01	64.2 $\pm$ 6.4	1.64
		CP	0.0311 $\pm$ 0.0032	2.04 $\pm$ 0.04	71.4 $\pm$ 8.2	1.82
	32	HMM	0.0351 $\pm$ 0.0030	2.31 $\pm$ 0.05	71.5 $\pm$ 6.7	1.82
		BTree	0.0318 $\pm$ 0.0040	2.27 $\pm$ 0.04	77.4 $\pm$ 8.7	1.98
16	1	FF	0.0299 $\pm$ 0.0023	1.72 $\pm$ 0.03	62.6 $\pm$ 4.3	1.60
		HMM	0.0400 $\pm$ 0.0045	2.32 $\pm$ 0.03	63.0 $\pm$ 6.3	1.61
	32	CP	0.0303 $\pm$ 0.0028	2.07 $\pm$ 0.01	73.9 $\pm$ 6.8	1.89
		BTree	0.0333 $\pm$ 0.0037	2.37 $\pm$ 0.05	76.9 $\pm$ 6.9	1.96

Table 15

			$\odot\mu_{\text{lat}} \downarrow$	$\boxtimes\mu_{\text{acc}} \uparrow$	$\mu_{\text{tok/s}} \uparrow$	$\boxtimes \times_{\text{STP}}$
1	STP	—	0.0256	—	39.2	1.00
8	FF	0	0.0269 $\pm$ 0.0001	1.73 $\pm$ 0.02	69.4 $\pm$ 0.9	1.77
		1	0.0323 $\pm$ 0.0041	1.95 $\pm$ 0.06	67.0 $\pm$ 9.4	1.71
		2	0.0308 $\pm$ 0.0003	2.02 $\pm$ 0.08	72.1 $\pm$ 3.1	1.84
		4	0.0318 $\pm$ 0.0001	2.18 $\pm$ 0.04	75.2 $\pm$ 1.0	1.92
	BTree	0	0.0301 $\pm$ 0.0004	2.31 $\pm$ 0.02	82.9 $\pm$ 0.6	2.12
		1	0.0334 $\pm$ 0.0021	2.54 $\pm$ 0.02	82.0 $\pm$ 5.8	2.09
		2	0.0332 $\pm$ 0.0011	2.71 $\pm$ 0.04	87.7 $\pm$ 1.9	2.24
		4	0.0362 $\pm$ 0.0042	2.82 $\pm$ 0.08	84.1 $\pm$ 10.1	2.15
16	FF	0	0.0273 $\pm$ 0.0002	1.73 $\pm$ 0.04	68.6 $\pm$ 0.6	1.75
		1	0.0300 $\pm$ 0.0004	1.94 $\pm$ 0.03	70.9 $\pm$ 1.3	1.81
		2	0.0308 $\pm$ 0.0002	2.00 $\pm$ 0.02	71.6 $\pm$ 0.1	1.83
		4	0.0323 $\pm$ 0.0005	2.12 $\pm$ 0.05	72.2 $\pm$ 1.2	1.84
	BTree	0	0.0312 $\pm$ 0.0004	2.42 $\pm$ 0.03	83.4 $\pm$ 1.4	2.13
		1	0.0329 $\pm$ 0.0006	2.63 $\pm$ 0.07	85.9 $\pm$ 3.3	2.19
		2	0.0343 $\pm$ 0.0002	2.65 $\pm$ 0.03	83.4 $\pm$ 1.2	2.13
		4	0.0376 $\pm$ 0.0041	2.76 $\pm$ 0.02	80.0 $\pm$ 8.3	2.04

K.2.2. GREEDY SPECULATIVE DECODING

Table 16

		$\odot\mu_{\text{lat}} \downarrow$	$\boxtimes\mu_{\text{acc}} \uparrow$	$\mu_{\text{tok/s}} \uparrow$	$\max_{\text{tok/s}}$
FF	1	0.0248 ± 0.0010	2.70 ± 0.00	111.8 ± 4.3	330.31
	8	0.0264 ± 0.0003	3.22 ± 0.03	128.2 ± 1.1	310.61
CP	16	0.0286 ± 0.0020	3.39 ± 0.05	125.2 ± 7.9	315.91
	32	0.0285 ± 0.0026	3.45 ± 0.04	128.4 ± 12.2	282.88

Table 17

			$\odot\mu_{\text{lat}} \downarrow$	$\boxtimes\mu_{\text{acc}} \uparrow$	$\mu_{\text{tok/s}} \uparrow$	$\boxtimes \times \text{STP}$
1	1	STP	0.0255	—	39.4	1.00
8	1	FF	0.0248 ± 0.0010	2.70 ± 0.00	111.8 ± 4.3	2.84
		CP	0.0285 ± 0.0026	3.45 ± 0.04	128.4 ± 12.2	3.26
	32	BTree	0.0283 ± 0.0022	3.72 ± 0.10	138.6 ± 7.7	3.52
		HMM	0.0295 ± 0.0023	3.94 ± 0.06	140.3 ± 11.5	3.56
16	1	FF	0.0257 ± 0.0027	2.73 ± 0.01	109.8 ± 11.2	2.79
		CP	0.0304 ± 0.0025	3.23 ± 0.06	112.7 ± 9.9	2.86
	32	HMM	0.0312 ± 0.0023	4.15 ± 0.07	139.6 ± 8.0	3.55
		BTree	0.0285 ± 0.0001	4.05 ± 0.06	149.8 ± 1.9	3.81

Table 18

			$\odot\mu_{\text{lat}} \downarrow$	$\boxtimes\mu_{\text{acc}} \uparrow$	$\mu_{\text{tok/s}} \uparrow$	$\boxtimes \times \text{STP}$
1	STP	—	0.0255	—	39.4	1.00
8	FF	0	0.0252 ± 0.0014	2.71 ± 0.00	110.8 ± 6.2	2.81
		1	0.0266 ± 0.0003	3.27 ± 0.03	126.8 ± 0.6	3.22
		2	0.0288 ± 0.0018	3.57 ± 0.04	128.4 ± 8.0	3.26
		4	0.0300 ± 0.0003	3.89 ± 0.06	134.8 ± 3.2	3.42
	BTree	0	0.0287 ± 0.0017	3.86 ± 0.07	141.7 ± 8.8	3.60
		1	0.0291 ± 0.0005	4.12 ± 0.05	148.7 ± 0.8	3.78
		2	0.0297 ± 0.0002	4.29 ± 0.03	151.7 ± 1.8	3.85
		4	0.0320 ± 0.0019	4.48 ± 0.05	147.0 ± 9.1	3.73
16	FF	0	0.0253 ± 0.0016	2.74 ± 0.01	111.2 ± 7.0	2.82
		1	0.0275 ± 0.0013	3.23 ± 0.05	121.3 ± 5.3	3.08
		2	0.0289 ± 0.0018	3.56 ± 0.04	127.8 ± 8.6	3.25
		4	0.0300 ± 0.0006	3.88 ± 0.05	134.2 ± 3.2	3.41
	BTree	0	0.0294 ± 0.0014	4.15 ± 0.04	148.8 ± 7.5	3.78
		1	0.0314 ± 0.0019	4.43 ± 0.10	148.7 ± 9.6	3.78
		2	0.0329 ± 0.0014	4.56 ± 0.05	146.2 ± 6.5	3.71
		4	0.0335 ± 0.0016	4.78 ± 0.02	150.5 ± 6.6	3.82

L. Results on RTX 3090 GPU

L.1. EvaByte

L.1.1. SPECULATIVE SAMPLING

Table 19

		$\odot\mu_{\text{lat}} \downarrow$	$\checkmark\mu_{\text{acc}} \uparrow$	$\mu_{\text{tok/s}} \uparrow$	$\text{max}_{\text{tok/s}}$
FF	1	0.0548	5.14	96.2	160.66
	8	0.0591	5.64	98.0	157.02
	16	0.0577	5.73	102.3	158.35
CP	32	0.0574	5.89	105.8	154.99
	64	0.0567	5.98	108.8	154.92
	128	0.0590	5.98	104.4	151.02

Table 20

			$\odot\mu_{\text{lat}} \downarrow$	$\checkmark\mu_{\text{acc}} \uparrow$	$\mu_{\text{tok/s}} \uparrow$	$\text{STP} \times$
1	1	STP	0.0472	—	21.2	1.00
		FF	0.0548	5.14	96.2	4.53
		HMM	0.0657	6.03	94.3	4.44
		BTree	0.0597	6.05	104.4	4.91
8	32	CP	0.0574	5.89	105.8	4.98
		FF	0.0556	5.37	99.2	4.67
		HMM	0.0774	6.88	91.2	4.29
		CP	0.0594	6.18	107.1	5.04
16	32	BTree	0.0639	6.71	108.2	5.09

Table 21

			$\odot\mu_{\text{lat}} \downarrow$	$\checkmark\mu_{\text{acc}} \uparrow$	$\mu_{\text{tok/s}} \uparrow$	$\text{STP} \times$
1	STP	—	0.0472	—	21.2	1.00
		0	0.0553	5.13	95.0	4.47
		1	0.0569	5.05	90.9	4.28
		2	0.0586	5.11	89.3	4.20
8	FF	4	0.0624	5.12	84.1	3.96
		0	0.0595	6.04	104.5	4.92
		1	0.0614	6.19	104.0	4.90
		2	0.0625	6.21	102.6	4.83
	BTree	4	0.0667	6.22	96.3	4.53
		0	0.0554	5.35	99.1	4.67
		1	0.0597	5.46	94.0	4.42
		2	0.0605	5.61	95.3	4.49
16	FF	4	0.0635	5.64	91.4	4.30
		0	0.0620	6.85	114.0	5.37
		1	0.0648	7.24	115.6	5.44
		2	0.0666	7.41	115.3	5.43
	BTree	4	0.0697	7.50	111.7	5.26

L.1.2. GREEDY SPECULATIVE DECODING

Table 22

		$\odot\mu_{\text{lat}} \downarrow$	$\boxtimes\mu_{\text{acc}} \uparrow$	$\mu_{\text{tok/s}} \uparrow$	$\text{max}_{\text{tok/s}}$
FF	1	0.0552	6.87	128.9	162.65
	8	0.0579	6.66	119.7	160.28
	16	0.0580	6.67	119.7	158.39
CP	32	0.0581	6.65	119.2	161.22
	64	0.0584	6.65	118.7	155.42
	128	0.0584	6.64	118.7	151.29

Table 23

			$\odot\mu_{\text{lat}} \downarrow$	$\boxtimes\mu_{\text{acc}} \uparrow$	$\mu_{\text{tok/s}} \uparrow$	$\boxtimes_{\text{STP}}$
1	1	STP	0.0466	—	21.5	1.00
8	1	FF	0.0552	6.87	128.9	5.98
		HMM	0.0603	6.69	115.3	5.35
	32	BTree	0.0589	6.66	117.7	5.46
		CP	0.0581	6.65	119.2	5.53
16	1	FF	0.0550	7.71	145.7	6.76
		HMM	0.0657	8.38	133.6	6.20
	32	CP	0.0602	7.70	134.3	6.23
		BTree	0.0621	8.13	137.5	6.38

Table 24

			$\odot\mu_{\text{lat}} \downarrow$	$\boxtimes\mu_{\text{acc}} \uparrow$	$\mu_{\text{tok/s}} \uparrow$	$\boxtimes_{\text{STP}}$
1	STP	—	0.0466	—	21.5	1.00
8	FF	0	0.0563	6.87	126.6	5.88
		1	0.0588	6.87	121.0	5.62
		2	0.0598	6.88	119.5	5.55
		4	0.0633	6.86	112.5	5.22
	BTree	0	0.0598	6.68	116.4	5.40
		1	0.0624	6.70	112.0	5.20
		2	0.0652	6.69	107.0	4.97
		4	0.0669	6.70	104.4	4.85
16	FF	0	0.0569	7.70	140.8	6.54
		1	0.0621	8.30	139.5	6.48
		2	0.0624	8.51	142.6	6.62
		4	0.0655	8.66	138.4	6.43
	BTree	0	0.0636	8.22	136.0	6.31
		1	0.0657	8.94	143.4	6.66
		2	0.0680	9.03	140.2	6.51
		4	0.0709	9.10	135.5	6.29

L.2. Llama 3.2 3B

L.2.1. SPECULATIVE SAMPLING

Table 25

		$\ominus\mu_{\text{lat}} \downarrow$	$\boxtimes\mu_{\text{acc}} \uparrow$	$\mu_{\text{tok/s}} \uparrow$	$\max_{\text{tok/s}}$
FF	1	0.0503	1.71	36.8	193.61
	8	0.0519	1.92	40.0	190.51
CP	16	0.0524	1.96	40.6	192.18
	32	0.0517	2.04	42.7	193.86

Table 26

			$\ominus\mu_{\text{lat}} \downarrow$	$\boxtimes\mu_{\text{acc}} \uparrow$	$\mu_{\text{tok/s}} \uparrow$	$\boxtimes_{\text{STP}}$
1	1	STP	0.0410	—	24.5	1.00
	1	FF	0.0503	1.71	36.8	1.51
8	32	HMM	0.0605	2.21	39.8	1.63
		CP	0.0517	2.04	42.7	1.75
	32	BTree	0.0554	2.19	42.9	1.76
		FF	0.0506	1.72	36.8	1.50
16	32	HMM	0.0703	2.25	34.8	1.42
		CP	0.0525	2.08	42.6	1.74
	32	BTree	0.0555	2.38	46.0	1.88
		FF	0.0506	1.72	36.8	1.50

Table 27

			$\ominus\mu_{\text{lat}} \downarrow$	$\boxtimes\mu_{\text{acc}} \uparrow$	$\mu_{\text{tok/s}} \uparrow$	$\boxtimes_{\text{STP}}$
1	STP	—	0.0410	—	24.5	1.00
		0	0.0497	1.71	37.4	1.53
8	FF	1	0.0533	1.94	40.0	1.64
		2	0.0538	2.05	41.8	1.71
		4	0.0568	2.11	40.9	1.67
		0	0.0540	2.36	47.1	1.93
	BTree	1	0.0566	2.56	48.8	1.99
		2	0.0577	2.68	50.1	2.05
		4	0.0604	2.71	48.6	1.99
		0	0.0495	1.71	37.5	1.53
16	FF	1	0.0534	1.91	39.5	1.61
		2	0.0546	2.04	41.0	1.68
		4	0.0573	2.05	39.8	1.63
		0	0.0562	2.40	46.2	1.89
	BTree	1	0.0583	2.60	48.3	1.97
		2	0.0596	2.65	48.1	1.97
		4	0.0620	2.74	47.7	1.95
		0	0.0495	1.71	37.5	1.53

L.2.2. GREEDY SPECULATIVE DECODING

Table 28

		$\odot\mu_{\text{lat}} \downarrow$	$\boxtimes\mu_{\text{acc}} \uparrow$	$\mu_{\text{tok/s}} \uparrow$	$\max_{\text{tok/s}}$
FF	1	0.0448	2.69	61.7	195.66
	8	0.0485	3.23	70.0	193.55
CP	16	0.0496	3.41	72.5	194.39
	32	0.0496	3.45	73.6	193.76

Table 29

			$\odot\mu_{\text{lat}} \downarrow$	$\boxtimes\mu_{\text{acc}} \uparrow$	$\mu_{\text{tok/s}} \uparrow$	$\boxtimes \times_{\text{STP}}$
1	1	STP	0.0413	—	24.3	1.00
	1	FF	0.0448	2.69	61.7	2.54
8	32	CP	0.0496	3.45	73.6	3.03
		BTree	0.0506	3.72	77.6	3.20
	32	HMM	0.0519	3.90	78.9	3.25
16	1	FF	0.0458	2.70	60.8	2.50
	32	CP	0.0500	3.20	68.0	2.80
		HMM	0.0559	4.10	77.1	3.17
	32	BTree	0.0518	4.00	81.7	3.36

Table 30

			$\odot\mu_{\text{lat}} \downarrow$	$\boxtimes\mu_{\text{acc}} \uparrow$	$\mu_{\text{tok/s}} \uparrow$	$\boxtimes \times_{\text{STP}}$
1	STP	—	0.0413	—	24.3	1.00
8	FF	0	0.0441	2.68	62.4	2.57
		1	0.0477	3.25	70.6	2.91
		2	0.0492	3.54	74.9	3.08
		4	0.0524	3.89	77.3	3.18
	BTree	0	0.0503	3.84	80.6	3.32
		1	0.0524	4.11	82.6	3.40
		2	0.0532	4.30	85.2	3.51
		4	0.0554	4.45	84.6	3.48
16	FF	0	0.0444	2.73	63.2	2.60
		1	0.0479	3.23	70.0	2.88
		2	0.0495	3.55	74.4	3.06
		4	0.0531	3.89	76.2	3.14
	BTree	0	0.0517	4.16	85.0	3.50
		1	0.0538	4.38	86.1	3.54
		2	0.0549	4.61	88.6	3.65
		4	0.0573	4.74	87.4	3.60

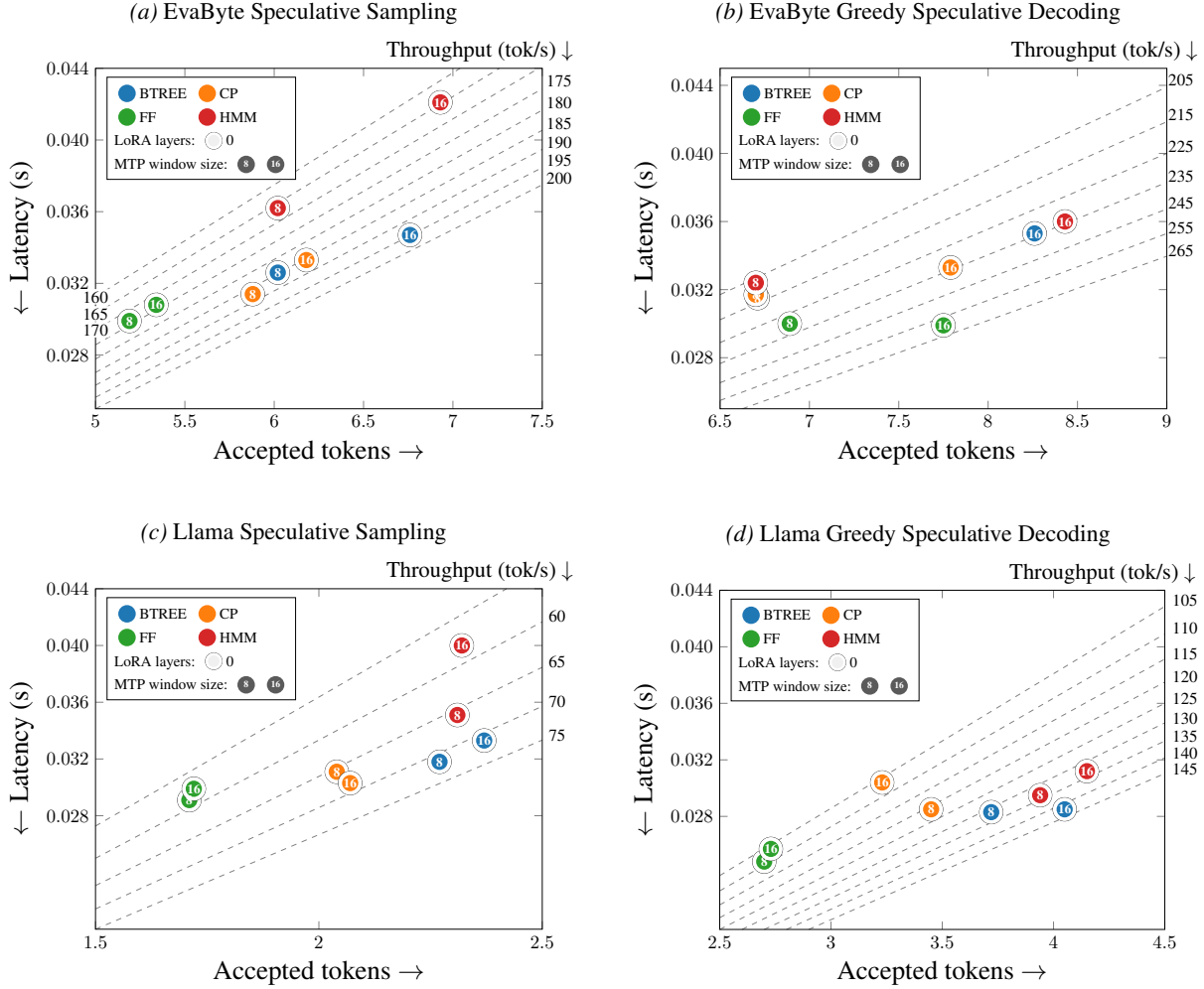
M. Further Plots on Extending the MTP Token Window to $n = 16$ 

Figure 6. Extending the MTP window to $n = 16$ is beneficial for models with no LoRA layers. Marker positions show the mean number of accepted tokens and the mean latency; iso-lines indicate the corresponding ratio of means, which may differ slightly from the per-run mean throughput reported in the tables above. More specifically, extending the MTP window of expressive PCs such as MTPC-HMM and MTPC-BTREE to $n = 16$ boosts their μ_{acc} in all settings: HMM 16 is to the right of HMM 8 and BTREE 16 is to the right of BTREE 8 in all four subplots. Importantly, MTPC-BTREE improves $\mu_{\text{tok/s}}$ for all cases apart from Evabyte with greedy speculative decoding (top right), highlighting its strong expressiveness–latency trade-off. We hypothesise that the reason MTPC-FF is so successful for EvaByte in the greedy setting is that this setup aligns with the objective optimised during pretraining because of the independence assumption. In contrast, although MTPC-HMM has a high μ_{acc} , it lags behind in throughput because of its high latency due to its AR nature (HMM 16 is much higher in the plots than HMM 8).

N. Further Plots on Adding more LoRA Layers to the Draft Model

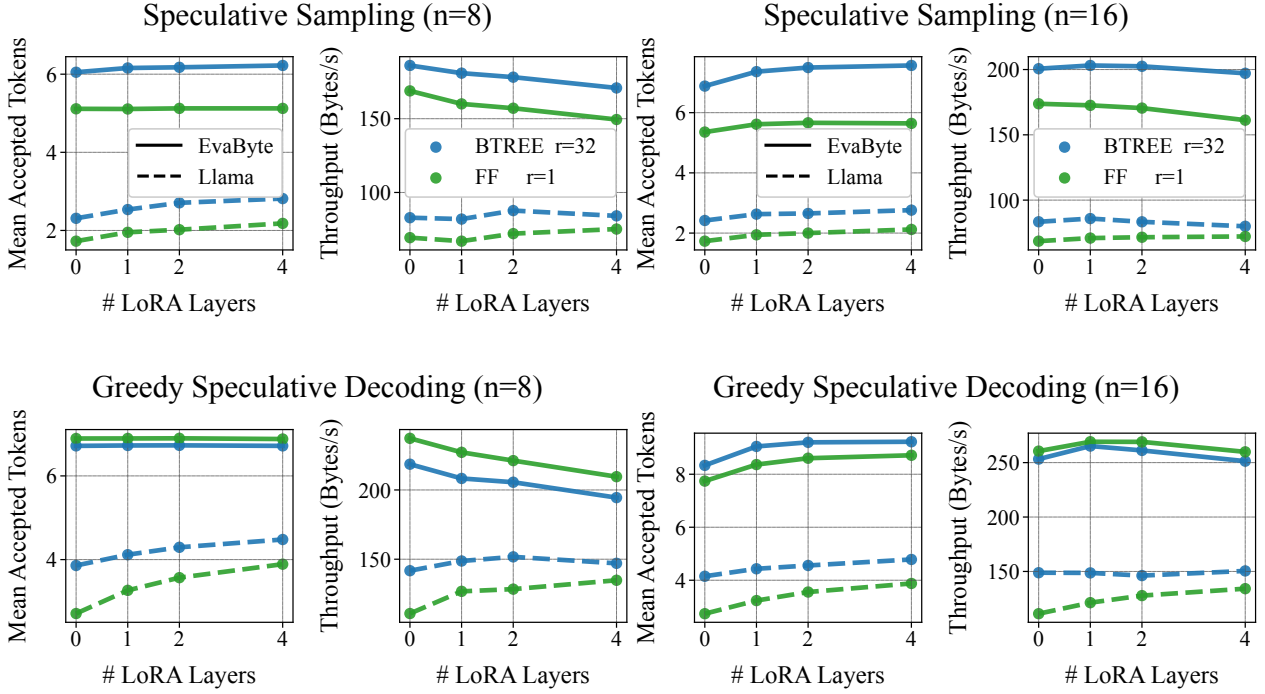


Figure 7. Increasing the expressiveness of our draft model by adding LoRA layers. For the $n = 8$ case, we get very little improvement for sampling with EvaByte and no improvement for greedy speculative decoding. We believe this is because EvaByte has been pretrained as a MTP model with $n = 8$ and as such the draft backbone has already converged to MTP representations which cannot be improved further. When we move to $n = 16$, EvaByte acceptance rates are boosted significantly both for speculative sampling and greedy speculative decoding and the optimal throughput is obtained with 1 LoRA layer. On the other hand, the Llama acceptance rates and throughput are boosted in all cases.